REFERENCE TEXT BOOKS

- ✓ **The complete Reference Java, 11th Edition, Herbert Schildt, TMH**

- ✓ **Joy with JAVA, Fundamentals of Object-Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2023.**

UNIT I:INTRODUCTION

Object Oriented Programming: Introduction, Program Structure in Java: Introduction, Writing Simple Java Programs, Elements or Tokens in Java Programs, Java Statements, Command Line Arguments, Escape Sequences, Comments.

Data Types and Variables: Introduction, Data Types in Java, Declaration of Variables, Scope of Variable Identifier, Literal Constants, Symbolic Constants, Static Variables and Methods, Attribute Final, Type Casting, User Input to Programs, Formatted Output with printf() Method.

Introduction to Operators: Precedence and Associativity of Operators, Assignment Operator (=), Basic Arithmetic Operators, Increment (++) and Decrement (--) Operators, Ternary Operator, Relational Operators, Boolean Logical Operators, Bitwise Logical Operators.

Control Statements: Introduction, if Expression, Nested if Expressions, if–else Expressions, Switch Statement, Iteration Statements, while Loop, do–while Loop, for Loop, Nested for Loop, For–Each, Break Statement, Continue Statement.

LEARNING OBJECTIVE

Make the student able to Understanding the Identify Java language components and how they work together in applications.

LEARNING OUTCOMES

After attending this session a student will be able to Learn the fundamentals of object-oriented programming in Java

Part-A- Object Oriented Programming

Concept -1 Introduction to JAVA



History of Java



Green Team.

Introduction of JAVA

✓Java is a high level, robust, object-oriented and secure programming language.

✓James Gosling had single moto while creating java i.e., Write in once and Run in any where

1) James Gosling, Mike Sheridan, and Patrick Naughton initiated the Java language project in June 1991. The small team of sun engineers called Green Team.

2) Originally designed for small, embedded systems in electronic appliances like set-top boxes.

3) Firstly, it was called "Green talk" by James Gosling, , and the file extension was .gt

4) After that, it was called Oakand was developed as a part of the Green project.

5) Why Oak? Oak is a symbol of strength and chosen as a national tree of many countries like U.S.A., France, Germany, Romania, etc.

6.)1995, Oak was renamed as "Java" because it was already a trademark by Oak Technologies.

7.) Java is an island of Indonesia where first coffee was produced (called java coffee).

8.) Java is just a name, not an acronym.

9.) Initially Java developed by James Gosling at Sun Microsystems and released in 1995.

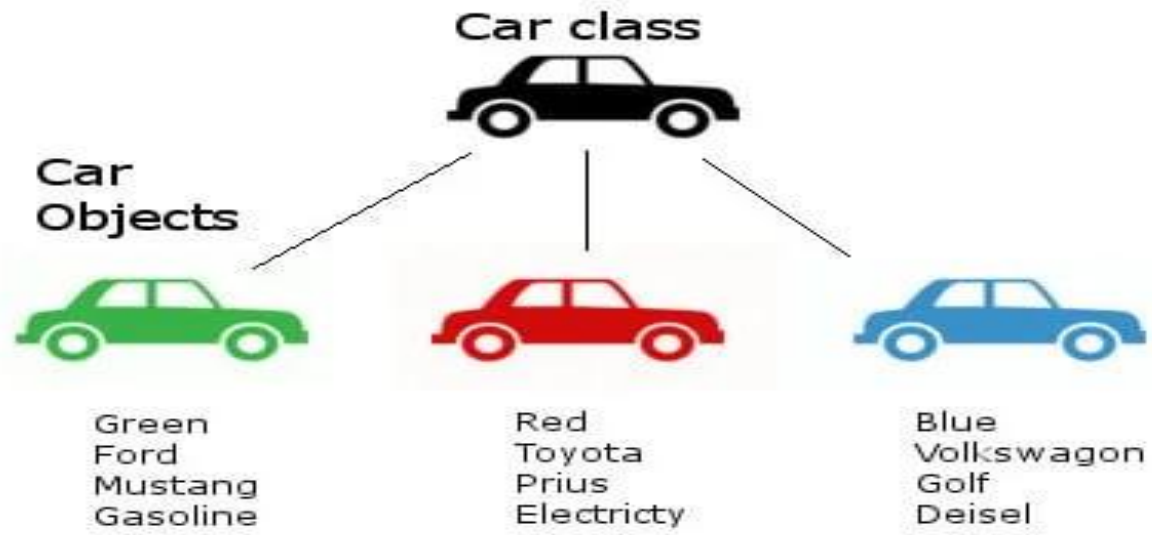
10.) On November 13, 2006, Sun released much of its Java virtual machine (JVM) as free and open-source software (FOSS).

11.) 2010 – Oracle acquired Sun Microsystems and became the steward of Java.

12.) On April 2, 2010, James Gosling resigned from Oracle.

13.) Java software runs on everything from laptops to data centers ,game consoled to super computers.

Example of Class and Object of JAVA



Colour
Brand
Model
FuelType

getCarDetails()
setCarDetails()

In Java, classes and objects form the foundation of Object-Oriented Programming (OOP).

- **A class is a blueprint used to create objects that share common properties and behaviour.**
- **An object is an instance of a class. It represents a specific entity created from the class template.**

Example : Main.java

APPLICATIONS OF JAVA

- ✓Desktop Applications such as acrobat reader, media player, antivirus, etc.
- ✓Web Applications such as irctc.co.in ,redbus ,bookmyshow
- ✓Enterprise Applications such as banking applications. (Flexcube ,Finacle)
- ✓Mobile Applications
- ✓Embedded System (ATM ,POC , Medical Devises , Smart TVs)
- ✓Smart Card (E-Visa , E-Passports , Metro Card etc)
- ✓Robotics (Manufacturing)
- ✓Games, etc.

Part-A- Object Oriented Programming

Concept-1: Concept of Object-Oriented Programming (OOP)

An object-oriented programming language generally supports five main features:

- **Classes**
- **Objects**
- **Encapsulation**
- **Polymorphism**
- **Inheritance**

Class :

- ✓ **Combine both data variables and methods into a single unit.**
- ✓ **Collection of objects is called class.**
- ✓ **It is a logical entity /template.**
- ✓ **A class can also be defined as a blueprint /template from which you can create an individual object.**
- ✓ **Class doesn't consume any space.**

Object :

- ✓ It is a real world entity i.e., physical entity
- ✓ An Object can be defined as an instance of a class.
- ✓ It is a run time entity
- ✓ An object contains an address and takes up some space in memory.
- ✓ For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

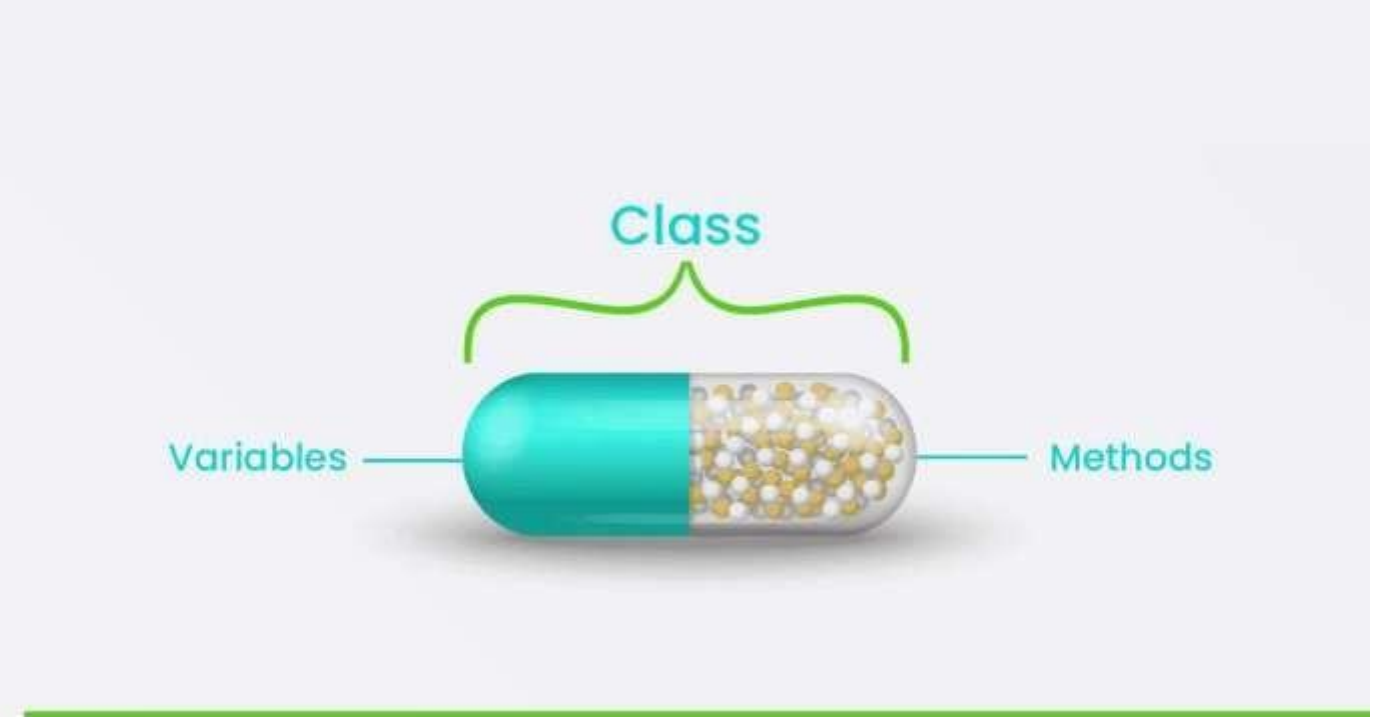
class	objects
Fruit	Apple Banana Mango

Another example:

class	objects
Car	Volvo Audi Toyota

Encapsulation

- ✓ **Binding (or) wrapping code and data together into a single unit are known as encapsulation.**
- ✓ **Data is stored within classes and it is not easy to access the data.**



Abstraction

Hiding internal details /implementation details and showing functionality /features is known as abstraction. i.e., we can show necessary information and hide unnecessary information/data.

In Java, abstraction is achieved using abstract classes and interfaces.

Polymorphism

If one task is performed in different ways, it is known as polymorphism. i.e., we can use same object in multiple place then we are getting multiple outputs.

Inheritance

- ✓ ***Acquiring properties of one class to another class it is known as inheritance.***
- ✓ ***It provides code reusability.***
- ✓ ***It is used to achieve runtime polymorphism.***

Part-A- Object Oriented Programming

Concept-2: Features of Java OR Buzz Words

The features of Java are also known as java *buzzwords*.

A list of most important features of Java language is given below.

1. **Simple**
2. **Object-Oriented**
3. **Portable**
4. **Platform independent**
5. **Secured**
6. **Robust**
7. **Architecture neutral**
8. **Interpreted**
9. **High Performance**
10. **Multithreaded**
11. **Distributed**
12. **Dynamic**

1. Simple:

Java is very easy to learn and its Syntax is simple, clean and easy to understand

2. Object-Oriented :

Java is an object-oriented programming language .

Everything in java is an object Java uses objects to organize and manipulate data making programs easier to understand and manage

3. Portable :

Java code can be easily run on different types of computers without any changes

4. Platform Independent:

Java code can be executed on multiple platforms, for example, Windows, Linux, Sun Solaris, Mac/OS, etc. i.e., Write Once and Run Anywhere (WORA).

5. Secured:

Java is best known for its security. With Java, we can develop virus-free systems.

Java is secured because:

- ✓ **No explicit pointer**
- ✓ **Java Programs run inside a virtual machine sandbox -JVM**

6.Robust:

Java makes an effort to check error at run time and compile time.

It uses a strong memory management system called garbage collector, Exception handling and auto garbage collection features .

7. Architecture neutral:

Compiler generates byte codes, which have nothing to do with a particular computer architecture, hence a Java program is easy to intrepret on any machine.

8. Interpreted:

Java supports cross-platform code through the Java bytecode.

Use of Bytecode can be interpreted on any platform by JVM.

9.High Performance:

Java is an interpreted language, so it will never be as fast as a compiled language like C or C++.

But Java enables high performance with the use of just-in-time compiler (JIT)

Java source code is first compiled into bytecode.
During execution, the JIT Compiler converts frequently used bytecode into native machine code, making execution much faster.

10. Multithreaded:

Java can run many tasks at the same time, which helps make programs faster and more efficient.

11. Distributed/Network-Friendly:

Java is distributed because it facilitates users to create distributed applications in Java.

This feature of Java makes us able to access files by calling the methods from any machine on the internet.i.e., Java is great for building programs that run over the internet or a network.

12. Dynamic/Flexible:

Java can adapt to different situations, like loading new code while your program is running.i.e., perform single compile and multiple execution

Part-A- Object Oriented Programming

Concept-2: Program Structure in Java

- ✓ **Documentation Section(optional)**
- ✓ **Package Declaration(optional)**
- ✓ **Import Statements**
- ✓ **Class Definition**
- ✓ **Main Method**

Documentation Section or Comment Section

The documentation section is an important section but optional for a Java program. It includes basic information about a Java program. The information includes the author's name, date of creation, version, program name, company name, and description of the program. Comments are non-executable parts of a program. A compiler does not execute a comment. It is used to improve the readability of the code.

To write the statements in the documentation section, we use comments. The comments may be **single-line, multi-line,** and **documentation** comments.

Single-line Comment: It starts with a pair of forwarding slash **(//).** For example:

//First Java Program

Multi-line Comment: It starts with a **/*** and ends with ***/.** We write between these two symbols.

For example:

Example of Caliculator.java

Package Declaration

The package declaration is optional. It is placed just after the documentation section. In this section, we declare the package name in which the class is placed.

Ex: com.rce.cse.java; //where rce is the package name

Import Statements

Many predefined classes are stored in packages in java.If we want to use any of them in our program we need to import that class. It is also optional

Ex: import java.util.Scanner; or import java.util.*;

Class Definition:

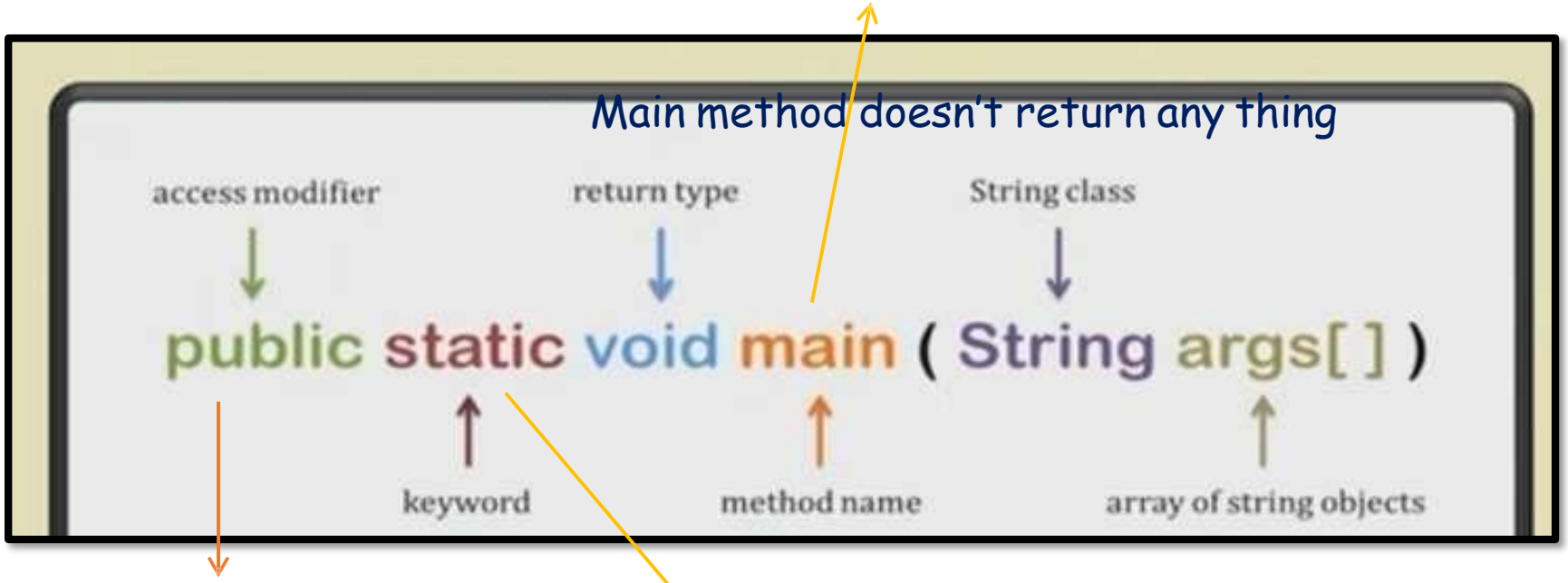
- ✓ A java program may contain several class definitions, classes are an essential part of any java program.
- ✓ It defines the information about the user-defined classes in a program.
- ✓ A class is a collection of variables and methods that operate on the fields.
- ✓ Every program in java will have at least one class with the main method.

Main Method :

- ✓ The main method is from where the execution actually starts and follows the order specified for the following statements.
- ✓ Let's take a look at a sample program to understand how it is structured.

Example:

```
// WRITE A JAVA PROGRAM TO DISPLAY Welcome  RCE CSE
class Sample {
   public static void main(String args[])
{
      System.out.println("Welcome to RCE CSE Department");
   }
}
```



- ✓ **Main method should be publicly usable**
- ✓ **Any one run a program if u want to restrict the program should not execute by any one then with draw public we use private**

To create an object to access main method but If you declare static then no object creation is not necessary

public static void main( )

- ✓ When the main method is declared public, it means that it can be used outside of this class as well.
- ✓ The word static means that we want to access a method without making its objects. As we call the main method without creating any object.
- ✓ The word void indicates that it does not return any value.
- ✓ The main is declared as void because it does not return any value. Main is the method, which is an essential part of any java program.

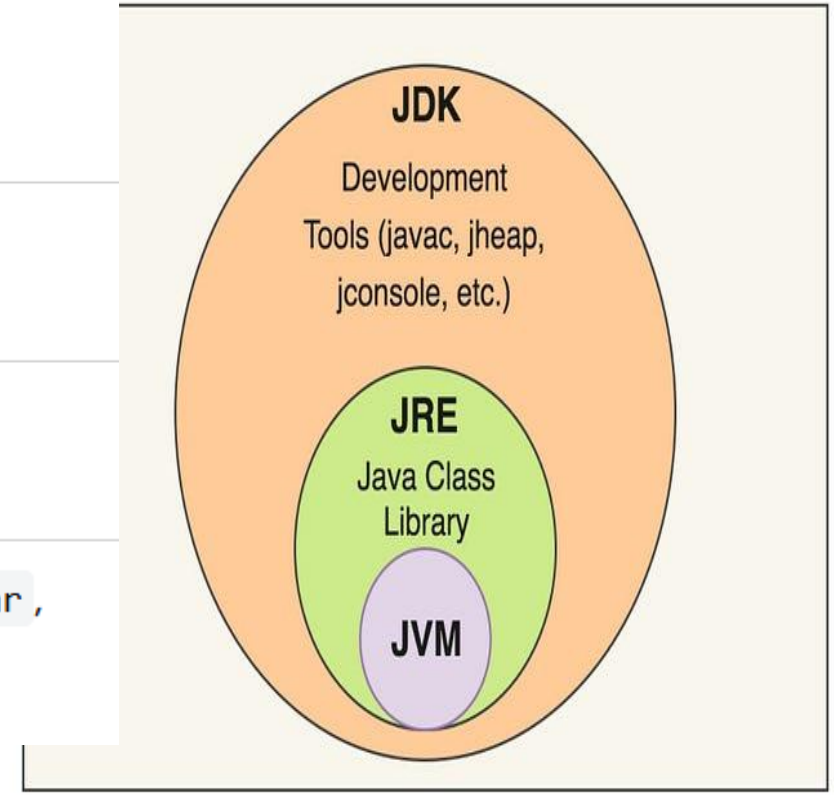
System.out.println( );

- ✓ **The statement is used to print the output on the screen where the system us predefined class, out is an object of the Print Writer class.**
- ✓ **The method println prints the next on the screen with a new line. All java statements end with a semicolon.**

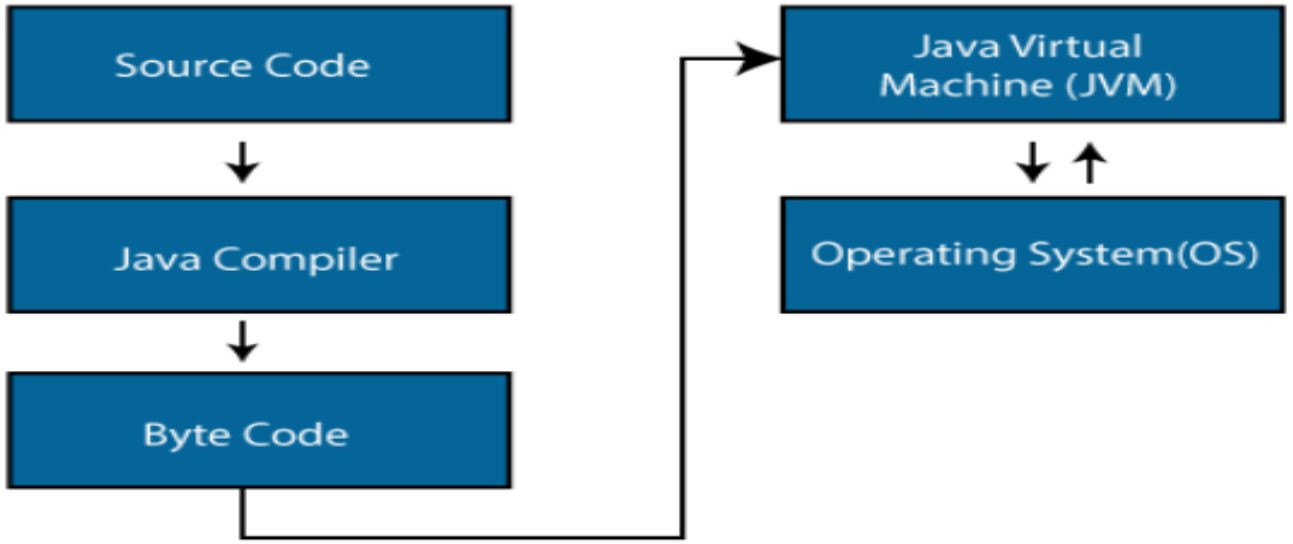
What is JDK,JRE,JVM

📊 Summary Comparison

Component	Target Audience	Primary Function	What is included?
JVM	Internal System	Executes bytecode line-by-line	Just-In-Time (JIT) compiler, Garbage collector
JRE	End Users	Runs Java software packages	JVM + Core libraries
JDK	Software Developers	Creates and compiles software	JRE + Tools like `javac`, `jar`, `javadoc`



Simple Flow of Java Program Execution



How Java Code is Converted and Executed

1. Write Java Source Code
2. Compile Java Code using javac (JDK)
3. Bytecode is Generated (Platform Independent)
4. JVM Loads and Executes Bytecode
5. JVM Converts Bytecode to Machine Code (if needed) and Executes
6. CPU Executes and Produces Output

Hello.java

```
class Hello {
 public static void
 main(String[] args) {
    int a = 10;
    int b = 20;
    int sum = a + b;
    System.out.println
    ("Sum = " + sum);
 }
}
```

→ **javac** — Java Compiler

→ **Hello.class** — Bytecode

```
10101010
00101010
10101010
00010101
10101010
...
```

→ **JVM (Java Virtual Machine)**

- Class Loader
- Bytecode Verifier
- Runtime Data Areas
- Execution Engine (Interpreter / JIT Compiler)
- Garbage Collector

→ **Machine Code (Native Code)**

10101010101010
010101010101

→ **CPU**

Output on Console

Sum = 30

Key Point: Java compiler **(javac)** does NOT generate machine code. It generates BYTECODE. JVM converts bytecode into machine code at runtime and executes it.

JDK, JRE and JVM Relationship

JDK (Java Development Kit)
Everything needed for Java Development

JRE (Java Runtime Environment)
Libraries + JVM to run Java programs

JVM (Java Virtual Machine)
Executes Java Bytecode

- Class Loader
- Runtime Data Areas
- Execution Engine (Interpreter / JIT)
- Garbage Collector

Development Tools

- javac
- jar
- javadoc
- javap
- debugger
- etc.

Detailed JVM Execution Flow

1. Class Loading
JVM loads the .class file using Class Loader Subsystem.

→ **2. Bytecode Verification**
JVM verifies the bytecode for safety and security.

→ **3. Execution**
JVM executes the bytecode using Interpreter and JIT Compiler.

→ **4. Native Code**
JIT converts hot code into native machine code (optimization).

→ **5. Output**
CPU executes the native code and produces output.

Important Concepts

- Bytecode is platform independent. (.class file)
- JVM is platform dependent (Windows JVM, Linux JVM, Mac JVM)
- "Write Once, Run Anywhere" is possible because of JVM and Bytecode.

Legend

- Source / Steps
- Tools / Compiler
- Bytecode
- JVM Components
- Execution / Output

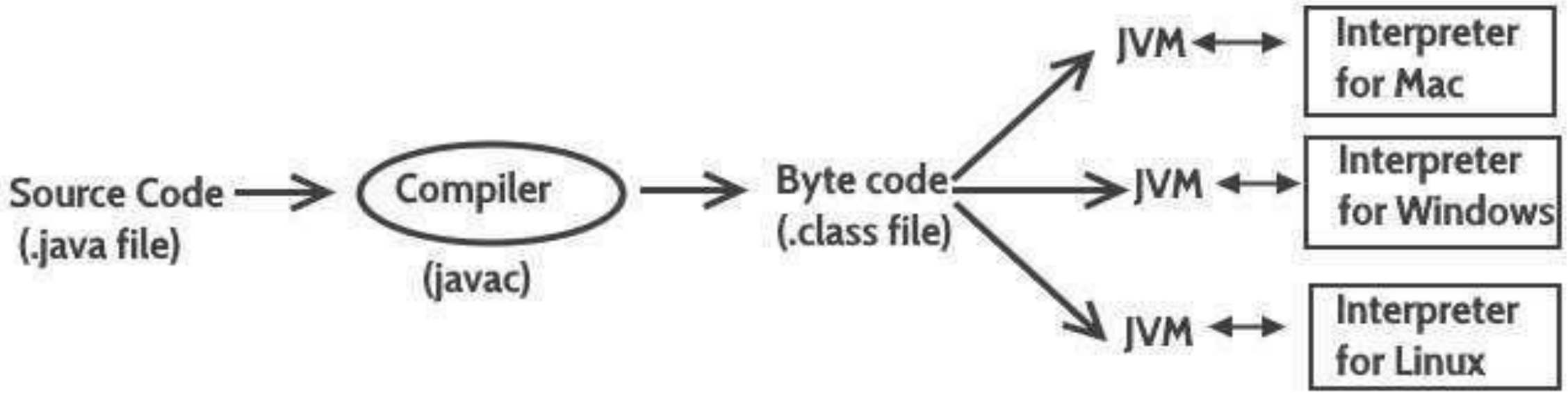
Part-A- Object Oriented Programming

Concept-5:JVM & Execution Process of Java Program

Java Virtual Machine (JVM)

- ✓ Java is a high level programming language. A program written in high level language cannot be run on any machine directly. First, it needs to be translated into that particular machine language.
- ✓ The **javac compiler** does this thing, it takes java program (.java file containing source code) and translates it into machine code (referred as byte code or .class file).
- ✓ Java Virtual Machine (JVM) is a virtual machine that resides in the real machine (your computer) and the **machine language for JVM is byte code**.
- ✓ This makes it easier for compiler as it has to generate byte code for JVM rather than different machine code for each type of machine.
- ✓ JVM executes the byte code generated by compiler and produce output.
- ✓ **J VM is the one that makes java platform independent**.

JUST IN TIME COMPILER(JIT)
OR
WRITE ONCE AND RUN ANYWHERE

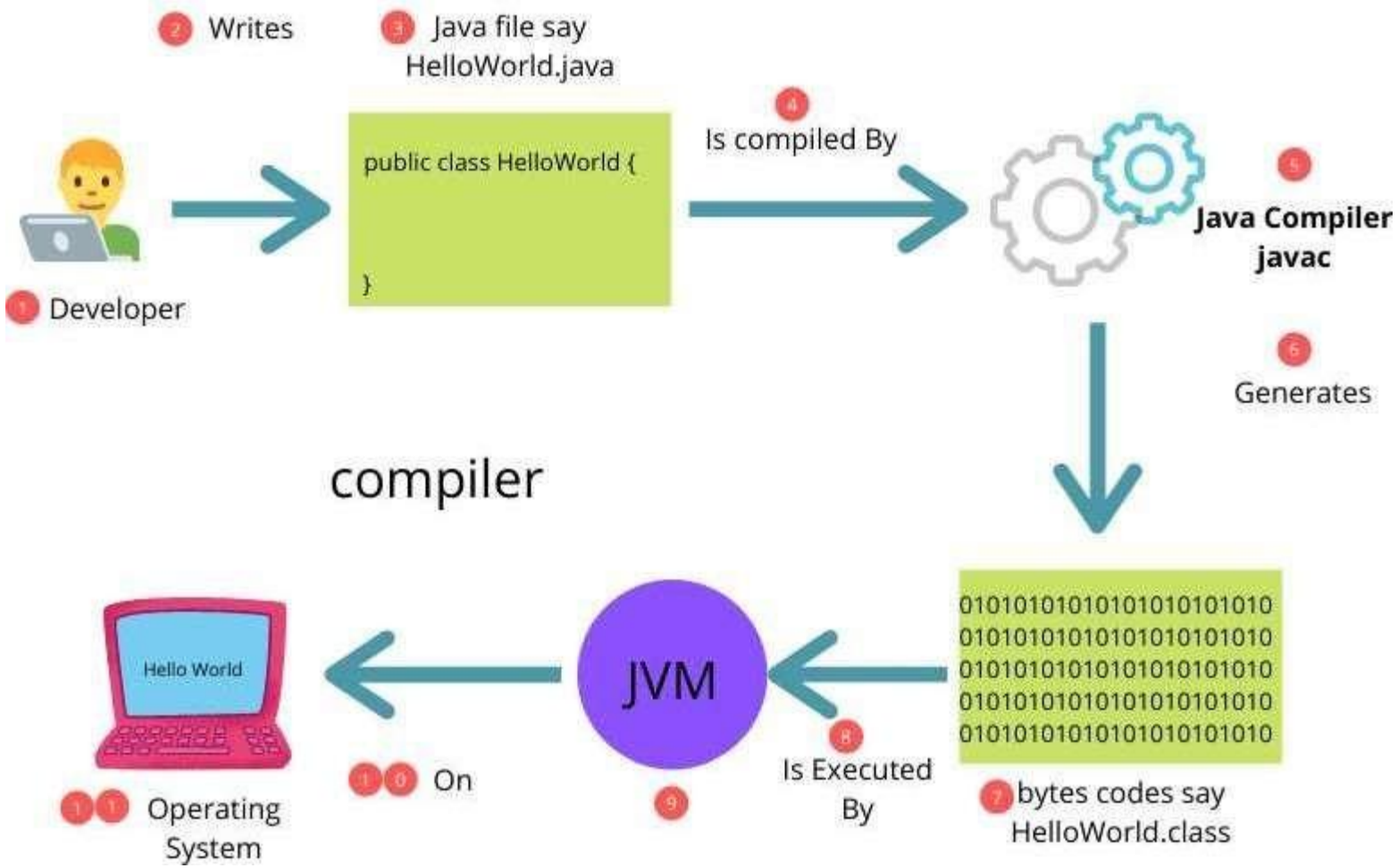


Execution Process of Java Program

Execution Process of Java Program The following three steps are

used to create and execute a java program.

- ✓ Create a source code (.java file).
- ✓ Compile the source code using javac command.
- ✓ Run or execute .class file uisng java command.

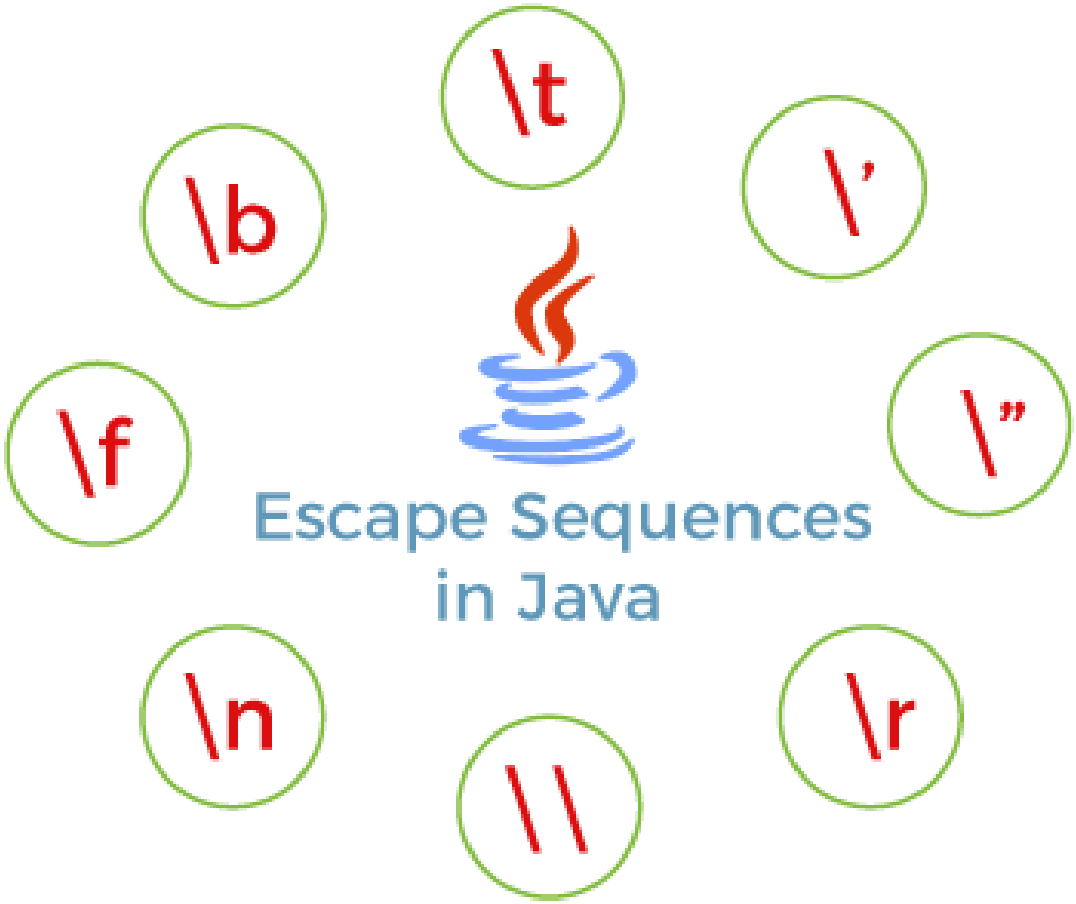

2 Writes
3 Java file say HelloWorld.java
4 Is compiled By
public class HelloWorld {
}
1 Developer
5 Java Compiler javac
6 Generates
compiler
0101010101010101010101010
0101010101010101010101010
0101010101010101010101010
0101010101010101010101010
0101010101010101010101010
7 bytes codes say HelloWorld.class
8 Is Executed By
JVM
9
10 On
Hello World
11 Operating System

Part-A- Object Oriented Programming

Concept-4: Java Escape Sequences

Java Escape Sequences

In Java, if a character is preceded by (\) id known as Java escape sequence or escape charter .



Escape Characters	Description
\t	It is used to insert a **tab** in the text at this point.
\'	It is used to insert a **single quote** character in the text at this point.
\"	It is used to insert a **double quote** character in the text at this point.
\r	It is used to insert a **carriage return** in the text at this point.
****	It is used to insert a **backslash character** in the text at this point.
\n	It is used to insert a **new line** in the text at this point.
\f	It is used to insert a **form feed** in the text at this point.
\b	It is used to insert a **backspace** in the text at this point.

Part-A- Object Oriented Programming

Concept-5: How to Get User Input from Console

Java **Scanner class** allows the user to take input from the console. **It belongs to java.util package.**

Java Scanner class is a text scanner that breaks the input into tokens using a delimiter. The delimiter is whitespace by default.

It is used to read the input of primitive types **like int, double, long, short, float, and byte.**

Importing the class is the first step.

```
import java.util.Scanner;  or  import java.util.*
```

The object of the class Scanner is declared as follows.

```
Scanner   sc= new Scanner (System. in);
```

Method	Description
int nextInt()	**Input as an integer.**
float nextFloat()	**Input as a float.**
double nextDouble()	**Input as a double.**
byte nextByte()	**Input as a byte.**
String nextLine()	**Input as string**
boolean nextBoolean()	**Input into a boolean value.**
long nextLong()	**Input as a long.**
short nextShort()	**Input as a Short.**

Example

```
import java.util.Scanner;
class Sum {
public static void main(String args[ ]) {
Scanner obj = new Scanner(System.in);
System.out.println("Enter a value:");
int a = obj.nextInt();
System.out.println("Enter b value:");
int b = obj.nextInt();
int result = a + b;
System.out.println("Display sum of two numbers: " + result);
}
}
```

Sampled output

```
D:\>javac Sum.java

D:\>java Sum
Enter a value:
11
Enter b value:
22
Display sum of two numbers: 33
```

Part-A- Object Oriented Programming

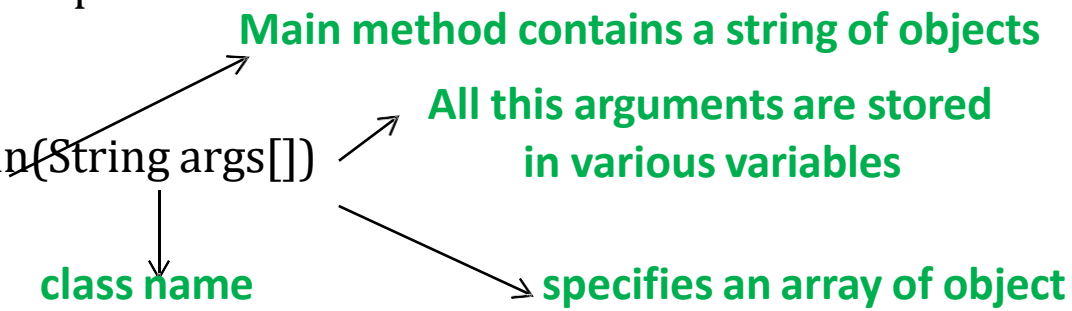
Concept-6: Command Line Arguments

- **The java command-line argument is an argument i.e. passed at the time running the java program.**
- **It is an information that directly follows the programs name on the command line when it is executed**
- **In command line argument always commonly used type object is String**
- **The arguments passed from the console can be received in the java program and it can be used as an input.**
- **So, it provides a convenient way to check the behavior of the program for the different values.**

You can pass N (1,2,3 and so on) numbers of arguments from the command prompt.

- **Simple example of command-line argument in java.**

```
class CommandLineExample
{
public static void main(String args[])
{
System.out.println("Your first argument is: "+args[0]);
}
}
```



1. compile by > javac CommandLineExample.java

2. run by > java CommandLineExample RCE EEE

Output: Your first argument is: RCE EEE

In this above example, we are receiving only one argument and printing it. To run this java program, you must pass at least one argument from the command prompt

```
class A
{
public static void main(String args[])
{
for(int i=0;i<args.length;i ++)
{
System.out.println(args[i]);
}
}
}
```

All this arguments are stored in various variables

Total Number of Arguments count

```
E:\java>javac A.java

E:\java>java A raj 1 2 3
raj
1
2
3
```

EXAMPLE

```
Class Add{
public static void main(String args[])
{
int a,b,c;
a=Integer.parseInt(args[0]);
b=Integer.parseInt(args[1]);
c=a+b;
System.out.println(c);
}
}
```

All this arguments are stored in various variables

Available in lang package

Class name

Method name

1. compile by > javac CommandLineExample.java
2. run by > java CommandLineExample 20 10

30

Part-A- Object Oriented Programming

Concept-7: Formatted Output in Java using printf( )

Formatted Output in Java using printf( )

Sometimes in programming, it is essential to print the output in a given specified format. Most users are familiar with the printf function in C. Let us discuss how we can Formatting Output with printf() in uses format specifiers for formatting.

There are certain data types are mentioned below:

- ✓ For Number Formatting by using %d
- ✓ Formatting Decimal Numbers by using %f
- ✓ For Boolean Formatting by using %b
- ✓ For String Formatting by using %s

Example

```
class formatedDemo {
public static void main(String args[]) {
int a=10;
double f=1.23; char ch='s';
String name="RamaChandra Engg College";
boolean b=true; System.out.printf("%d\n",a);
System.out.printf("%f\n",f);
System.out.printf("%c\n",ch);
System.out.printf("%s\n",name);
System.out.printf("%b\n",b);
}
}
```

Part-A- Object Oriented Programming

Concept-8: Variables in Java & Keywords

Keywords in Java

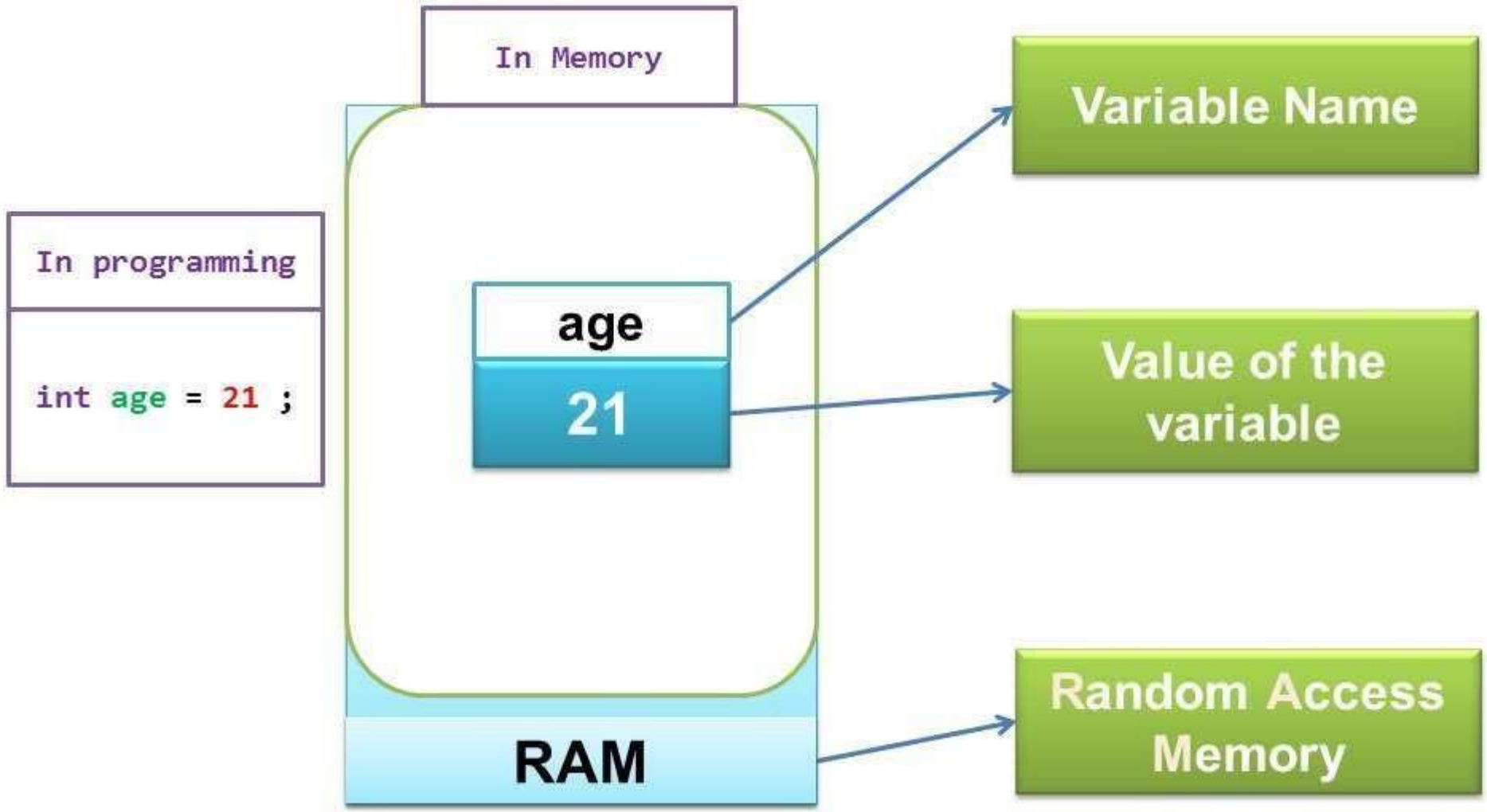
- ✓A keyword is a reserved word that conveys special meaning to the Java compiler.
- ✓These are reserved for special purposes and we cannot use them as identifier names otherwise the compiler will give an error.
- ✓In Java there are 51 keywords are used in Java. All of them have different purposes and meanings.
- ✓Reserve words/Key words which are available in language package and these all are known by compiler

Keywords In Java

1. abstract
2. assert
3. boolean
4. break
5. byte
6. case
7. catch
8. char
9. class
10. continue
11. default
12. do
13. double
14. else
15. enum
16. extends
17. final
18. finally
19. float
20. for
21. if
22. implements
23. import
24. instanceof
25. int
26. interface
27. long
28. native
29. new
30. package
31. private
32. protected
33. public
34. return
35. short
36. static
37. strictfp
38. super
39. switch
40. synchronized
41. this
42. throw
43. throws
44. transient
45. try
46. void
47. volatile
48. while

Variables in Java

- ✓ A **variable** is a container which holds the value while the Java program is executed.
- ✓ A variable is assigned with a **data type**.
- ✓ Variable is a name of **memory location**.
- ✓ Variable is name of **reserved area allocated in memory**. In other words, it is a name of memory location. It is a combination of **"vary + able"** that means its value can be changed.
- ✓ All variable must be declared before they can be used in java.
- ✓ A variable is defined by the combination of an **variable name, a type, and an optional initializer**


In Memory
In programming
int age = 21 ;
age
21
RAM
Variable Name
Value of the variable
Random Access Memory

RULES FOR DEFINING VARIABLES

- ✓ Variable name must not exceed 31 character
- ✓ Variable name can have alphabets, digits, currency character ($) and underscore(_).
- ✓ A variable name can start with the alphabet, currency character ($) and underscore(_) only.
- ✓ It can't start with by a digit.
- ✓ No whitespace is allowed within the variable name.
- ✓ Except underscore(__) and currency character ($) no other special symbols are used as variable name
- ✓ A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid variable names:

- ✓ `int a;`
- ✓ `Int _ab;`
- ✓ `int a30;`
- ✓ `int a_b;`

Invalid variable names:

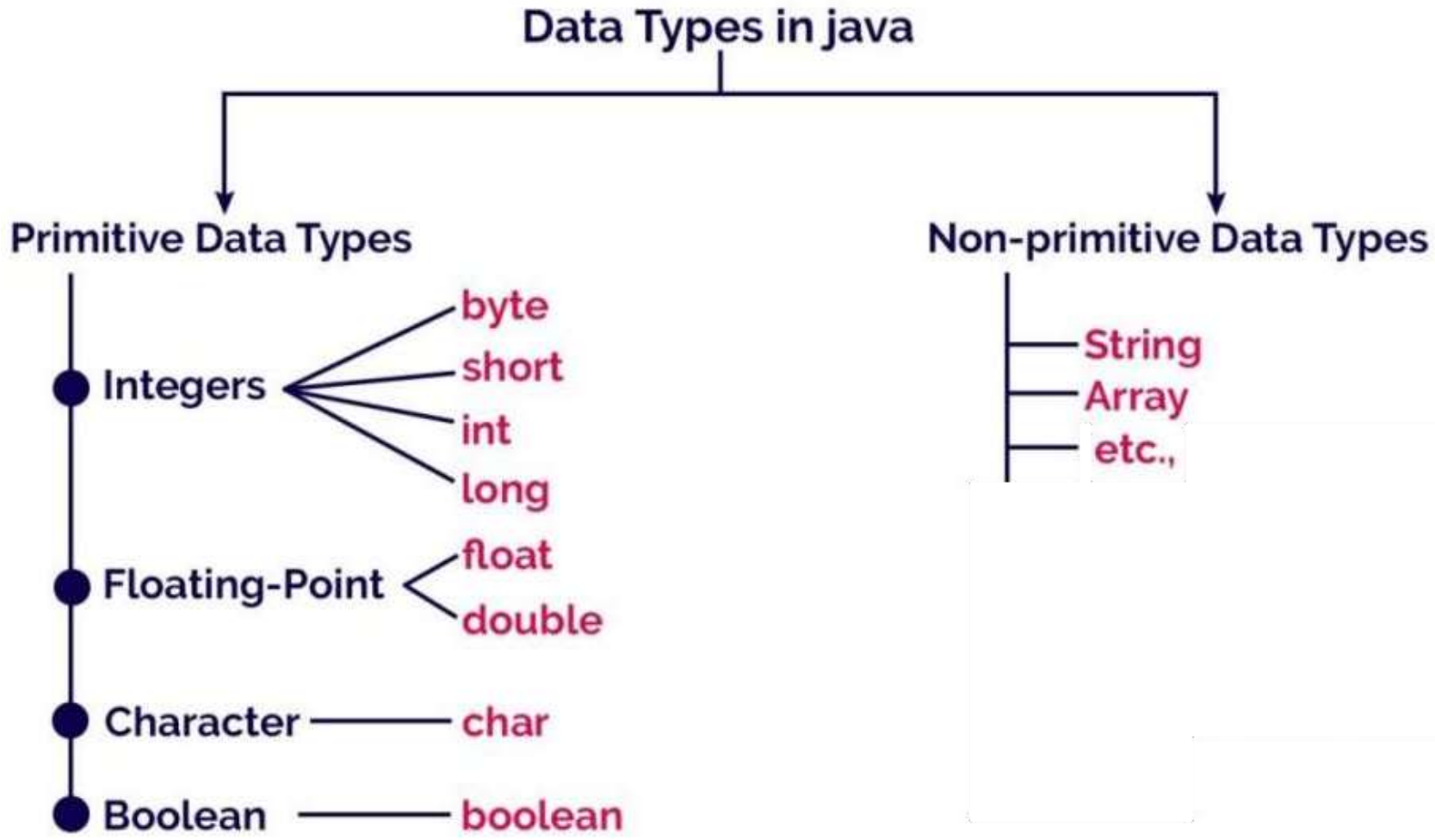
- ✓ `int 2;`
- ✓ `Int a b;`
- ✓ `int if;`

Part-A- Object Oriented Programming

Concept-10: Data Types in Java

Data Types in Java

✓ **Data types specify the different sizes and values that can be stored in the variable.**



✓ There are two types of data types in Java:

I. Primitive data types:

✓ The primitive data types are **built-in data types** and they specify the type of value stored in a variable and the memory size.

✓ The primitive data types include boolean, char, byte, short, int, long, float and double.

II. Non-primitive data types:

✓ Non-primitive data types are the reference data types or user-created data types.
✓ Non-primitive data type include Arrays and Strings

Difference Between Primitive and Non-Primitive Data Type

Primitive Data Types	Non-Primitive Data Types
Store actual values	Store references to objects
Predefined by Java	Created by the programmer or provided by Java
Fixed size	Size depends on the object
Cannot be null	Can be null
Examples: int, char, float, boolean	Examples: String, Arrays, Classes, Objects, Interfaces, Enums

1.Boolean Data Type :

- ✓ The Boolean data type is used to store only two possible values:
- ✓ This data type is used for simple flags that track True/False
- ✓ The Boolean data type specifies one bit of information,but its "size" can't be

  exact.(1-bit)
- ✓ Boolean data type variable are used in the dealing
- ✓ Its default value is false.
- ✓ Example: boolean one = false

2.Char Data Type

- ✓ **The char data type is a single 16-bit Unicode character. That is 2 byte.**
- ✓ **Its value-range lies between '\u0000' (or) to '\uffff'.**
- ✓ **The char data type is used to store characters.**
- ✓ **Example: char letter = 'A'**

Integers Number

- ✓ Integer are whole number that is, they represent number that do not have a fractional part.
- ✓ The integer can be declared in four types according to the size of memory allocated.

a. Byte b. Short c. Int d. Long

a.Byte Data Type

- ✓ The **byte** data type is an example of primitive data type.
- ✓ It is an **8-bit signed integer**. That is **1 byte**
- ✓ Its value-range lies between **-128 to 127**.
- ✓ Its minimum value is -128 and maximum value is 127.
- ✓ **Its default value is 0.**
- ✓ **Example : byte a=10 , byte b-20**

b.Short Data Type

- ✓ The **short** data type is a **16-bit signed integer**. That is **2 byte.**
- ✓ Its value-range lies between -32,768 to 32,767 (inclusive).
- ✓ Its minimum value is **-32,768 and maximum value is 32,767**.
- ✓ **Its default value is 0.**

Example: **short s = 10000, short r = -5000**

c.Int Data Type

- ✓ The **int** data type is a **32-bit signed integer**. That is **4 byte.**
- ✓ Its value-range lies between **-2,147,483,648 or (-2^31) to 2,147,483,647 or (2^31 -1).**
- ✓ Its minimum value is -2,147,483,648 and maximum value is 2,147,483,647.
- ✓ **Its default value is 0.**

Example: int a = 100000, int b = -200000

d.Long Data Type

- ✓ The **long** data type is a **64 bit signed integer. That is 8 byte.**
- ✓ The value-range lies between **(-2^63) to (2^63 -1)**
- ✓ Its minimum value is **(-2^63)** and maximum value is **(2^63 -1).**
- ✓ **Its default value is 0.**
- ✓ **Example: long a=123455334453, long b=3123453211343**

4.Float point number

- ✓ The numbers that are not whole numbers or those that have fractional part, Examples are 3.141, 476.6, and so on.

- ✓ Java supports two types of such numbers

a.Float Data Type

- ✓ The float data type is a **32-bit floating point.** That is **4 byte**

- ✓ Its value range is 1.4 e-45 to 3.4e+38

- ✓ This is used for single-precision floating point number that is 7 digits after the decimal point

b.Double Data Type

- ✓ The double data type is a **64-bit floating point.** That is 8 **byte.**
- ✓ Its value range is 4.9e-324 to 1.8e+308
- ✓ **This is used for double-precision floating point number that is 15 digits after the decimal point.**
- ✓ The double data type also should never be used for precise values, such as currency.
- ✓ **Its default value is 0.0d.**

Example: double d1 = 12.3456789.

Literal constants, Symbolic constants

- **Literal Constant:** A constant value written directly in the source code, such as 25, 3.14, 'A', "Hello", or true.
- **Symbolic Constant:** A named constant declared using the final keyword whose value remains unchanged throughout the program, such as final double PI = 3.14159;.

Elements or Tokens in Java

Java tokens are the smallest individual units of a Java program recognized by the compiler. The five types of Java tokens are **keywords, identifiers, literals, operators, and separators.**

Token	Description	Examples
Keywords	Reserved words with predefined meaning	int, class, if, return
Identifiers	Names of variables, methods, classes	age, Student, display()
Literals	Fixed values written directly in code	10, 3.14, "Java", 'A'
Operators	Symbols used to perform operations	+, -, *, =, &&
Separators	Symbols used to separate program elements	{}, (), ;, ,, []

Part-B Object Oriented Programming

Concept-12: Scope of Variable Identifier,

Static Variables and Methods

Types of Variables

There are three types of variables in Java

1. local variable 2. instance variable 3. static variable

Local Variable

- ✓ A variable declared inside the body of the method is called **local variable.**
- ✓ You can use this variable only within that method and the other methods in the class.
- ✓ A local variable cannot be defined with **"static"** keyword.

Static variable

- ✓A variable which is declared as static is called **static variable.** It cannot be local.
- ✓You can create a single copy of static variable and share among all the instances of the class.
- ✓Memory allocation for static variable happens only once when the class is loaded in the memory.

Instance variable

- ✓A variable declared inside the class but outside the body of the method, is called **instance variable**.
- ✓It is not declared as static.
- ✓It is called **instance variable** because its value is **instance specific** and is **not shared among instances.**

```
class Variable
{
 int data=50; //instance variable
static int m=100; //static variable
public static void main(String args[])
{
 int n=90; //local variable
Variable obj=new Variable(); //object creation
System.out.println(obj.data);
System.out.println(m);
System.out.println(n);
}
}
```

```
F:\JAVA LAB_AI&ML\3>javac Variable.java

F:\JAVA LAB_AI&ML\3>java Variable
50
100
90
```

Static Method in Java With Examples

- ✓ Any method that uses the static keyword is referred to as a static method.

Features of static method:

- ✓ A static method in Java is a method that is part of a class rather than an instance of that class.
- ✓ Every instance of a class has access to the method.
- ✓ Static methods have access to class variables (static variables) without using the class's object (instance).
- ✓ Only static data may be accessed by a static method. It is unable to access data that is not static (instance variables).
- ✓ In both static and non-static methods, static methods can be accessed directly.

Type casting in Java

Typecasting in Java is the process of converting one data type to another data type using the casting operator. When you assign a value from one primitive data type to another type, this is known as type casting.

Syntax:

```
datatype  variableName = (datatype)value;
```

// Java program to demonstrate Widening Type Casting and Narrow

Constants

- ✓ In Java, a symbolic constant is a named constant value defined once and used throughout a program. Symbolic constants are declared using the final keyword.
- ✓ Which indicates that the value cannot be changed once it is initialized.
- ✓ The naming convention for symbolic constants is to use all capital letters with underscores separating words.

Syntax of Symbolic Constants

final datatype CONSTANTNAME = value;

final: The final keyword indicates that the value of the constant cannot be changed once it is initialized.

data_type: The data type of the constant such as int, double, boolean, or String.

CONSTANTNAME: The name of the constant which should be written in all capital letters with underscores separating words.

value: The initial value of the constant must be of the same data type as the constant.

Example: `final double PI = 3.14159;`

Example:

```
class Circlearea
{
   // Declare a symbolic constant for PI
static final double PI = 3.14159;
public static void main(String args [ ])
{
double radius = 5.0;
// Calculate the area of the circle using the constant PI
double area = PI * radius * radius;
// Display the results
System.out.println("Area: " + area);
}
}
```

```
C:\javap>javac Circlearea.java

C:\javap>java Circlearea
Area: 78.53975
```

Part-C: Operators

Concept-01: OPERATORS IN JAVA

OPERATORS IN JAVA

Operator in java is a symbol that is used to perform operations.

There are many types of operators in Java which are given below:

1. **Assignment Operator**
2. **Arithmetic Operator**
3. **Increment & decrement Operator**
4. **Ternary Operator**
5. **Relational Operator**
6. **Boolean Logical Operator**
7. **Bitwise Logical Operator**

1.Assignment Operator

- ✓ Assignment operator (**=**) is used to assign a value to any variable.
- ✓ It has right-to-left associativity, i.e. value given on the right-hand side of the operator is assigned to the variable on the left, and therefore right-hand side value must be declared before using it or should be a constant.
- ✓ The general format of the assignment operator is:

✓ **Datatype variablename = value;**

- ✓ In many cases, the assignment operator can be combined with other operators to build a shorter version of the statement called a **Compound Statement**. For example, instead of **a = a+5, we can write a += 5**.
- ✓ **+=,** for adding the left operand with the right operand and then assigning it to the variable on the left.
- ✓ **-=,** for subtracting the right operand from the left operand and then assigning it to the variable on the left.
- ✓ ***=,** for multiplying the left operand with the right operand and then assigning it to the variable on the left.
- ✓ **/=,** for dividing the left operand by the right operand and then assigning it to the variable on the left.
- ✓ **%=,** for assigning the modulo of the left operand by the right operand and then assigning it to the variable on the left.

2. Arithmetic operators:

✓ In java, arithmetic operators are used to performing basic mathematical operations like addition, subtraction, multiplication, division, modulus,etc.,

Operator	Meaning	Example
+	Addition	10 + 5 = 15
-	Subtraction	10 - 5 = 5
*	Multiplication	10 * 5 = 50
/	Division	10 / 5 = 2
%	Modulus - Remainder of the Division	5 % 2 = 1

3. Increment/Decrement operators

- ✓**These operators are used to add 1 to the variable (increment) and subtract 1 from**

The variable (decrement).

- ✓**These operators can be applied in two forms. .**

1. Pre-Increment

In pre-increment, the increment operator is used as the prefix. The increment operation is performed immediately

Example

result = ++a;

The above expression can be expanded as

a = a + 1;

result = a;

2. Post-Increment

In post-increment, the increment operator is used as the suffix of the operand. The increment operation is performed after all the other operations are done.

Example

result = a++;

The above expression is equivalent

result =a;

a = a + 1;

1. Pre-Decrement Operator

The pre-decrement operator decreases the value of the variable immediately when encountered.

Example

```
result = --m;
which can be expanded to
m = m - 1;
result = m;
```

2. Post-Decrement Operator

The post-decrement happens when the decrement operator is used as the suffix of the variable. In this case, the decrement operation is performed after all the other operators are evaluated.

Example

```
result = m--;
The above expression can be expanded as
result = m;
 m = m-1;
```

4. Ternary Operator (?) or Conditional Operator:

In this operator, first, we verify a condition, If the condition is TRUE the first option is performed, if the condition is FALSE the second option is performed.

Condition ? Truevalue : falsevalue ;

For example,

```
Eligible=(age >= 18) ? "Can Vote" : "Cannot Vote";

System.out.println(Eligible);
```

5.. Relational operators:

- ✓ The relational operators are the symbols that are used to compare two values. That means the relational operators are used to check the relationship between two values. Every relational operator has two possible results either TRUE or FALSE.
- ✓ The relational operators used are <=, >, >=, ==, !=.

S.no	Operators	Example	Description
1	>	x > y	x is greater than y
2	<	x < y	x is less than y
3	>=	x >= y	x is greater than or equal to y
4	<=	x <= y	x is less than or equal to y
5	==	x == y	x is equal to y
6	!=	x != y	x is not equal to y

6.Boolean Logical Operator:

The logical operators are the symbols that are used to combine multiple conditions into one condition. The following table provides information about logical operators

Boolean logical operators operate only on boolean operands. These operators combine one or two boolean values to form a new boolean depending upon the operation. Logical operators produce results or outputs in the form of boolean values i.e., either true or false.

&& (logical and)	Called Logical AND operator. If both the operands are True, then the condition becomes True.	(A && B)
|| (logical or)	Called Logical OR Operator. If any of the two operands are True then the condition becomes True.	(A || B)
! (logical not)	Called Logical NOT Operator. Use to reverses the logical state of its operand. If a condition is true then Logical NOT operator will make false.	!(A && B)

- not

x	not x
False	True
True	False

- and

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

- or

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True



7.Bitwise Operators :

- ✓ The bitwise operators are used to perform bit-level operations in the java programming language.
- ✓ When we use the bitwise operators, the operations are performed based on binary values.

1. Bitwise OR (|)

This operator is a binary operator, denoted by '|'. It returns bit by bit OR of input values, i.e., if either of the bits is 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise OR Operation of 5 and 7
  0101
| 0111
 ________
  0111  = 7 (In decimal)
```

2. Bitwise AND (&)

This operator is a binary operator, denoted by '&.' It returns bit by bit AND of input values, i.e., if both bits are 1, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise AND Operation of 5 and 7
  0101
& 0111
 ________
  0101  = 5 (In decimal)
```

3. Bitwise XOR (^)

This operator is a binary operator, denoted by '^.' It returns bit by bit XOR of input values, i.e., if corresponding bits are different, it gives 1, else it shows 0.

Example:

```
a = 5 = 0101 (In Binary)
b = 7 = 0111 (In Binary)

Bitwise XOR Operation of 5 and 7
  0101
^ 0111
 ________
  0010  = 2 (In decimal)
```

4. Bitwise Complement (~)

This operator is a unary operator, denoted by '~.' It returns the one's complement representation of the input value, i.e., with all bits inverted, which means it makes every 0 to 1, and every 1 to 0.

Example:

```
a = 5 = 0101 (In Binary)

Bitwise Complement Operation of 5

~ 0101
 ________
  1010  = 10 (In decimal)
```

Part-C: Operators

Concept-02: OPERATOR PRECEDENCE AND ASSOCIATIVITY IN JAVA

OPERATOR PRECEDENCE AND ASSOCIATIVITY IN JAVA

✓The concept of **operator precedence and associativity** helps in determining which operators will be given priority when there are multiple operators in the expression.

✓It is very common to have multiple operators in Java and the compiler first evaluates the operator with higher precedence.

Category	Operators	Associativity
Postfix	++ - -	Left to right
Unary	+ - ! ~ ++ - -	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left

Operator Precedence

Operator precedence determines which operation is performed first in an expression with more than one operator with different precedence.

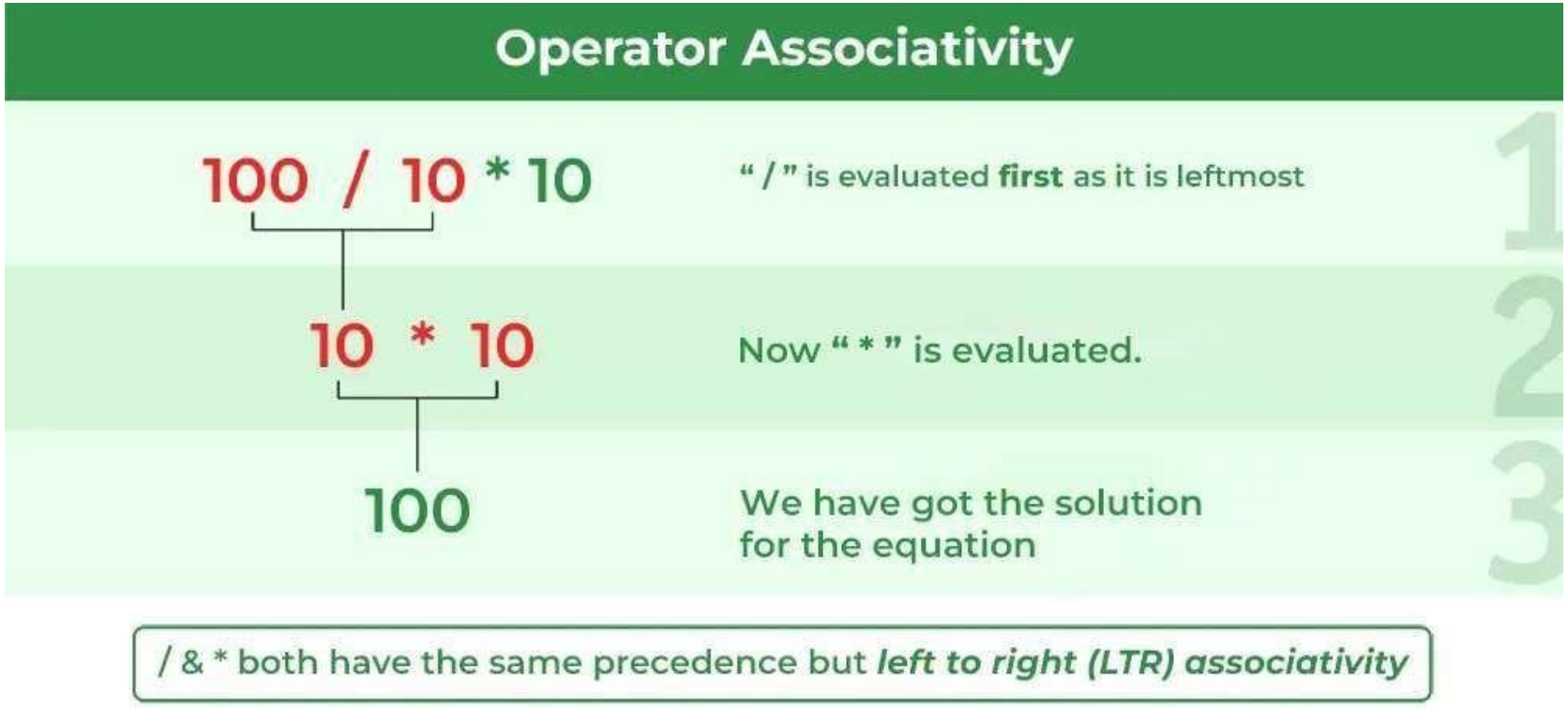
5 * 6 – 2 / 4 +1-0 = 4

2 % 1 – 4 * 4 + 1 *4 = -30

Operator Associativity

Operator associativity is used when two operators of the same precedence appear in an expression.

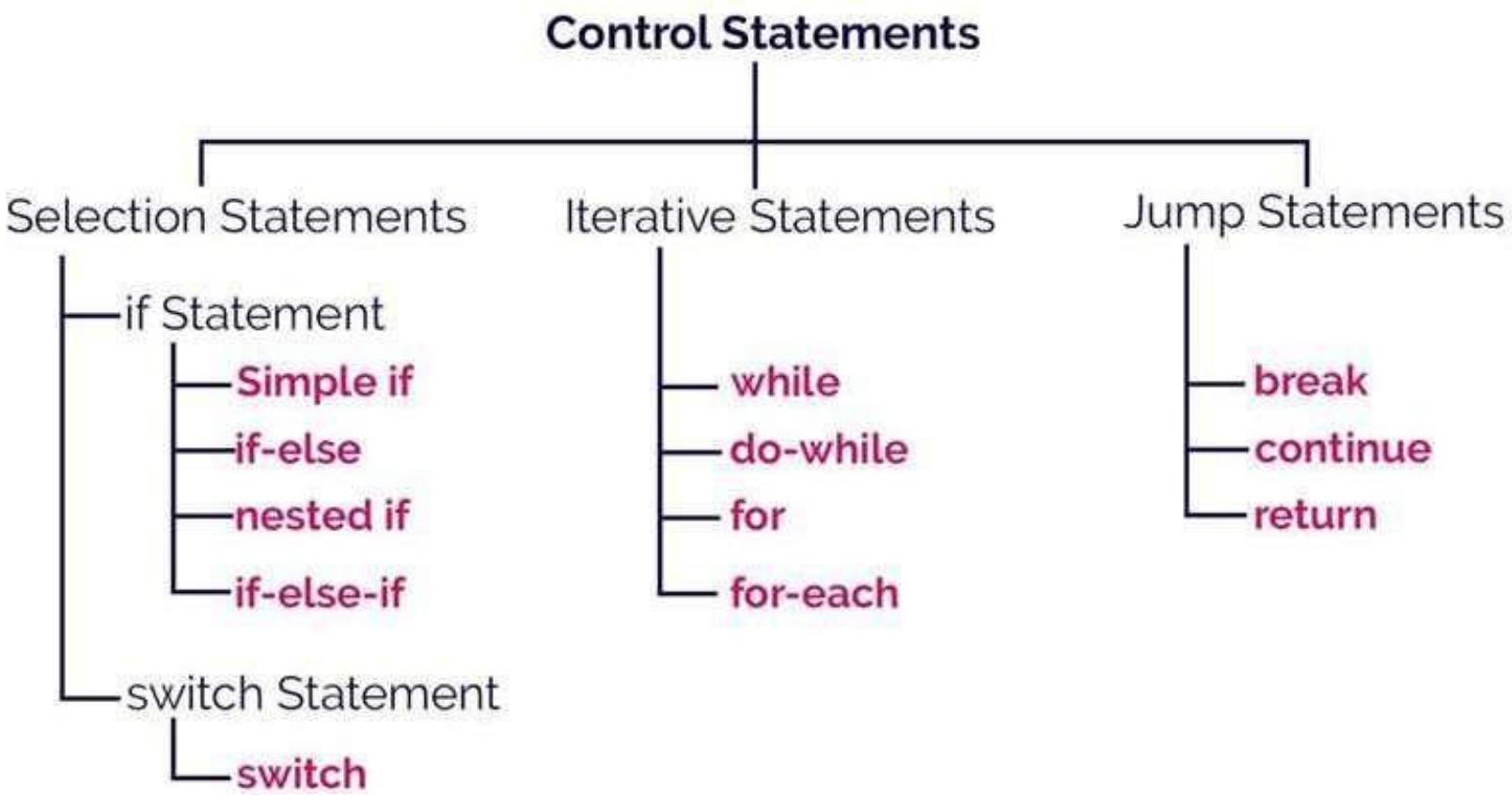
Associativity can be either from **Left to Right or Right to Left.**



Part-D: Control Statements

Concept-1: Control Statements in Java

Concept-1: Control Statements in Java



1.Decision control Structure

- Based on decision the control flow in only one direction
- Here are some most commonly used control structures:

- ✓ **Simple if**
- ✓ **if-else**
- ✓ **Nested if**
- ✓ **if-else-if**

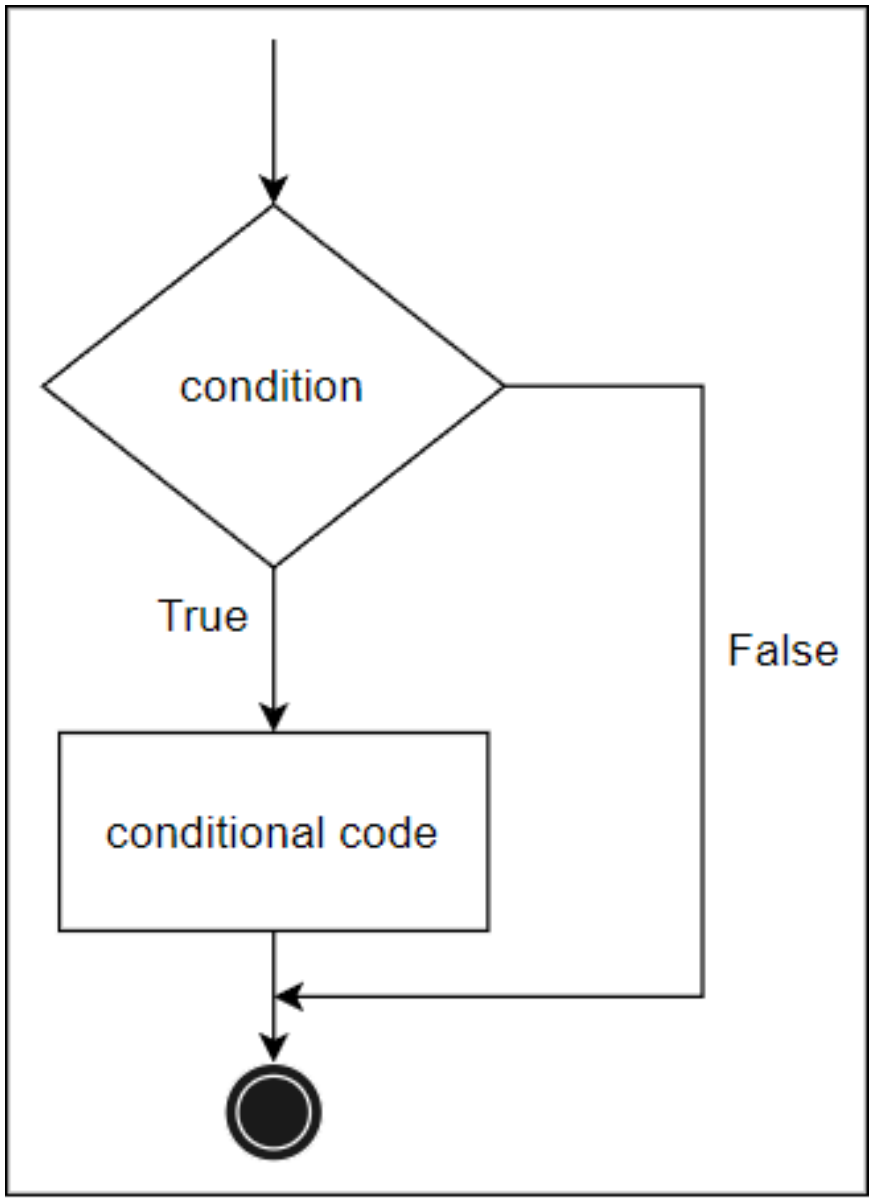
A.Simple if statement:

if statement is the simplest decision-making statement. It is used to decide whether a certain statement or block of statements will be executed or not i.e., if a certain condition is true then a block of statements is executed otherwise not.

Syntax:

```
if(condition)
{
Statement block ;
}
```

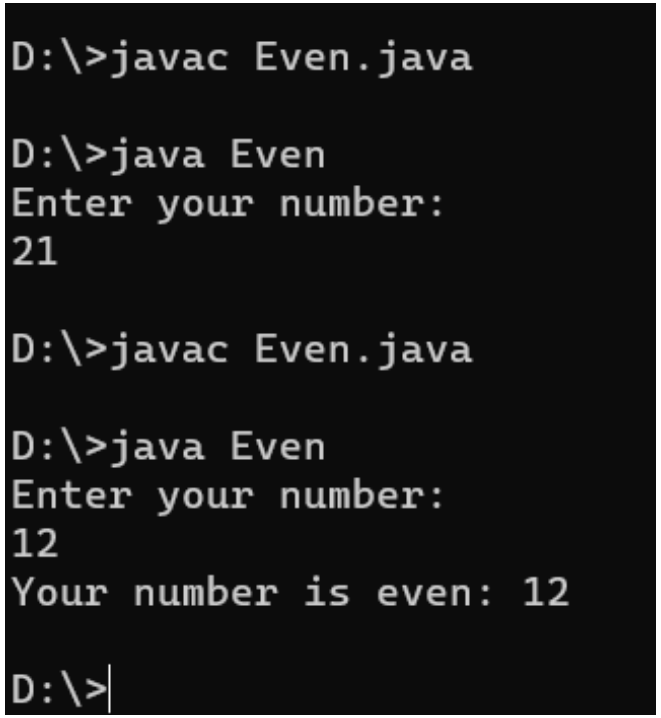
flowchart



Example:

Write a java program to find whether a given number is even or not using Simple if decision statement

```
import java.util.Scanner;
class Even
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter your number:");
int number = sc.nextInt();
if (number % 2 == 0)
{
System.out.println("Your number is even: " + number);
}
}
}
```



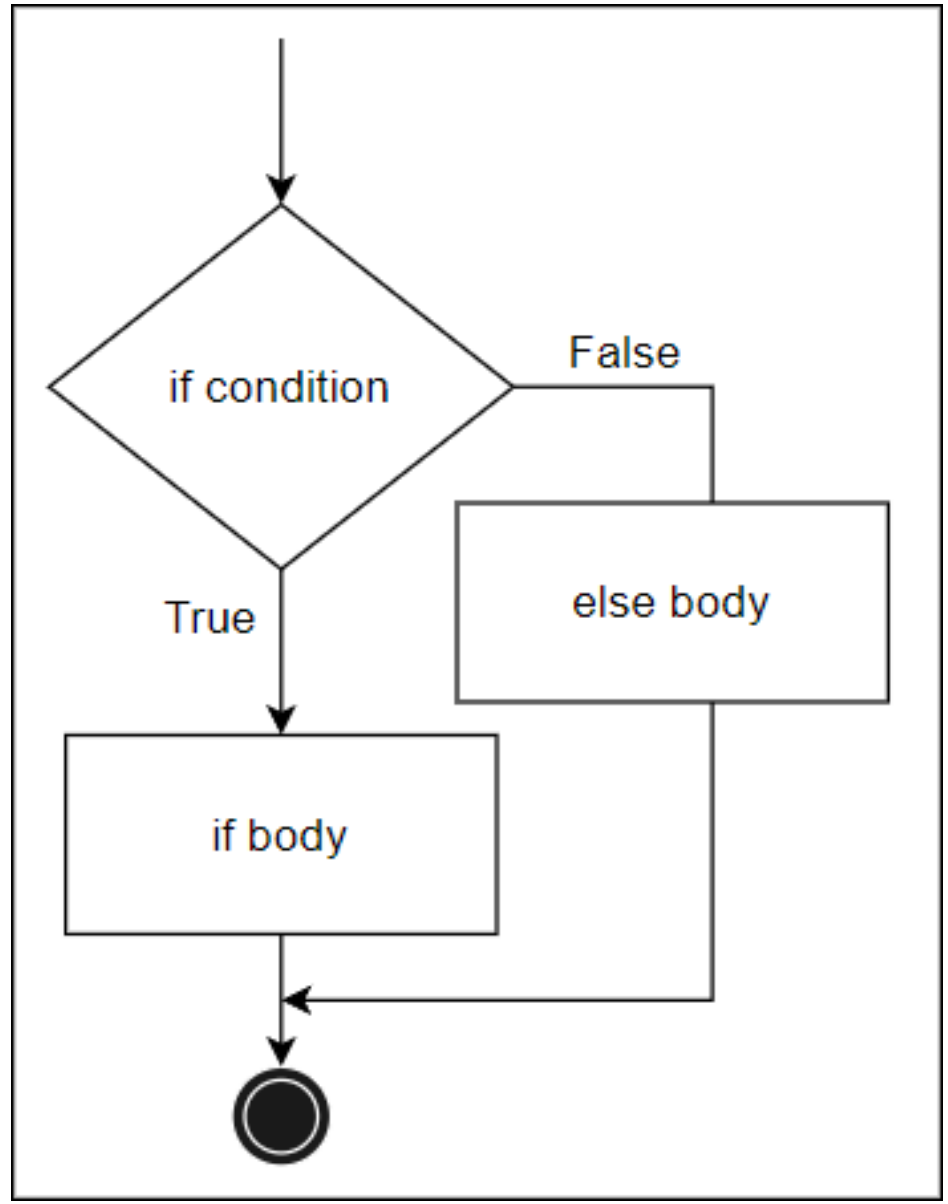
B.if else statement:

The if-else statement evaluates the condition and will execute the body of if if the test condition is True, but if the condition is False, then the body of else is executed.

Syntax:

```
if(condition)
{
Statement block;
}
else
{
Statement block;
}
```

flowchart



Example:

Write a java program to find whether a given number is even or not using Simple if decision statement

```
import java.util.Scanner;
class Even
{
public static void main(String[] args)
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter your number:");
int number = sc.nextInt();
if (number % 2 == 0)
{
System.out.println("Your number is even: " + number);
}
else
{
System.out.println("Your number is odd: " + number);
}
}
```

```
D:\>javac Even.java

D:\>java Even
Enter your number:
12
Your number is even: 12

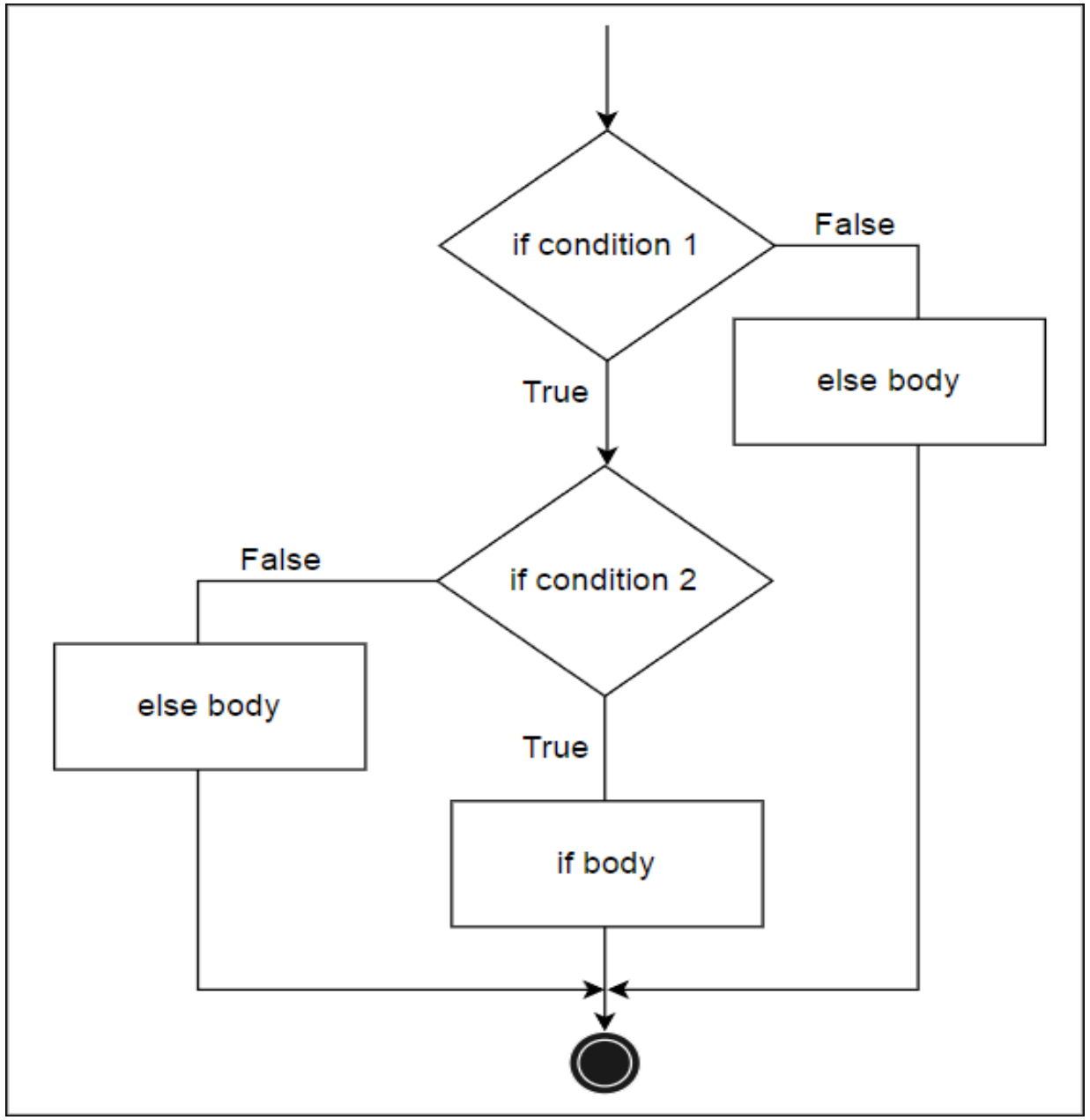
D:\>java Even
Enter your number:
21
Your number is odd: 21
```

c.Nested if statements : The nested if statement represents the *if block within another if block*. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
if(condition 1)
 {
     if(condition 2)
     {
         statement 1; //executes when condition 1
&2 is true
     }
     else
     {
     statement 2; //executes when condition 2 is false
     }
}
else
{
statement 3: //executes when both conditions are false
```

flowchart


if condition 1
False
else body
True
if condition 2
False
else body
True
if body

```
import java.util.*;
class ConditonalDemo
{
public static void main(String args[])
{
Scanner sc=new Scanner(System.in);
System.out.println("enter n1");
int n1=sc.nextInt();
System.out.println("enter n2");
int n2=sc.nextInt();
System.out.println("enter n3");
int n3=sc.nextInt();
if(n1>n2)
{
        if(n1>n3)
        {
        System.out.println(n1 + "is big");
        }
        else
        {
        System.out.println(n3 + "is big");
        }
}
else
{
System.out.println(n2 + "is big");
}
}
}
```

```
C:\javap>javac A.java

C:\javap>java A
enter n1
10
enter n2
20
enter n3
5
20is big
```

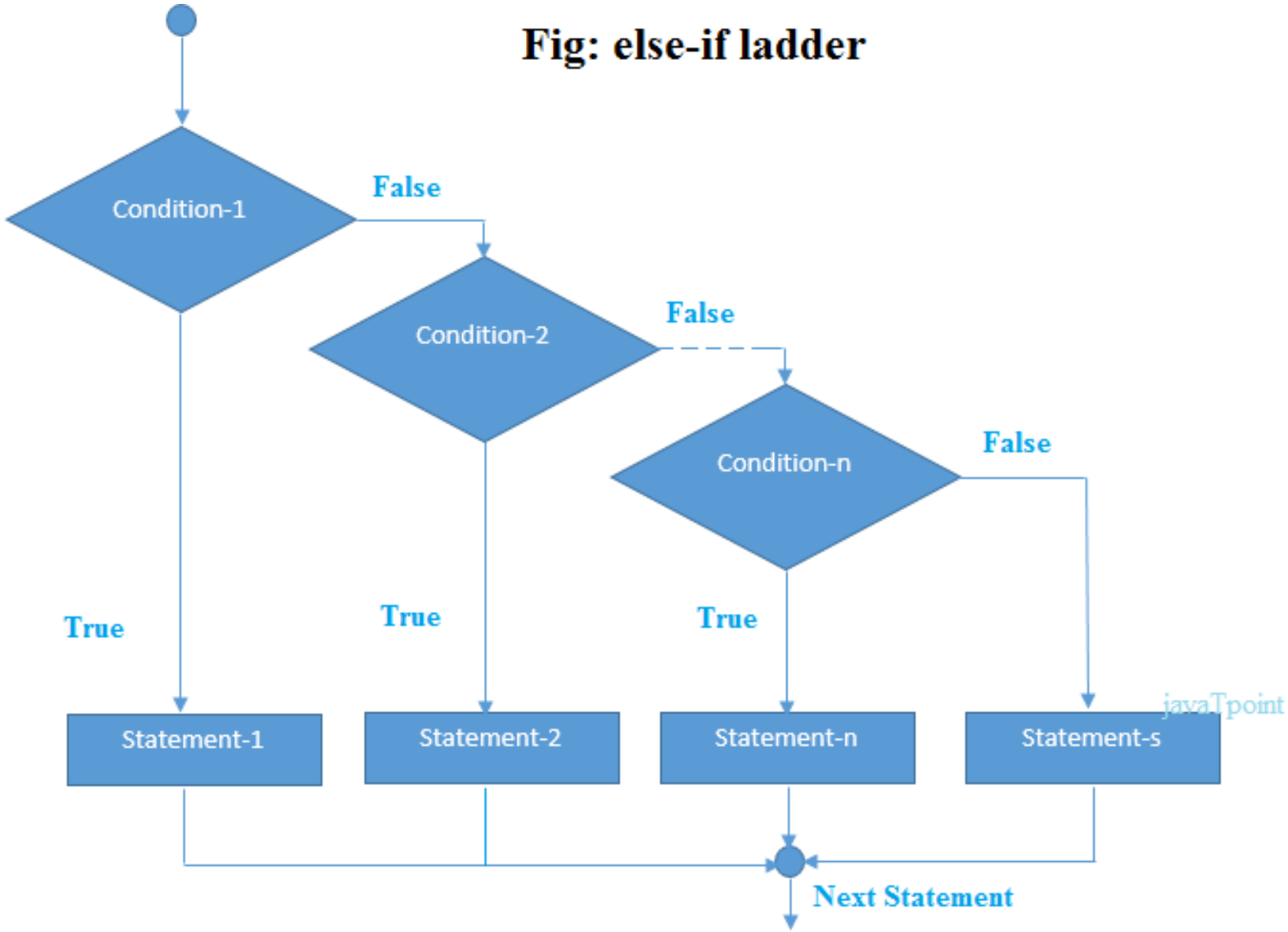
d.if-else-if ladder:

Here, a user can decide among multiple options.The if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that 'if' is executed, and the rest of the ladder is bypassed. If none of the conditions is true, then the final else statement will be executed. There can be as many as 'else if' blocks associated with one 'if' block but only one 'else' block is allowed with one 'if' block.

Syntax:

```
if(condition 1)
 {
statement 1;  //executes when condition 1 is true
}
else if(condition 2)
{
statement 2; //executes when condition 2 is true
}
else
{
statement 2; //executes when all the conditions are fa
lse
}
```

flowchart


Fig: else-if ladder
Condition-1
False
Condition-2
False
Condition-n
False
True
True
True
Statement-1
Statement-2
Statement-n
Statement-s
javaTpoint
Next Statement

Example

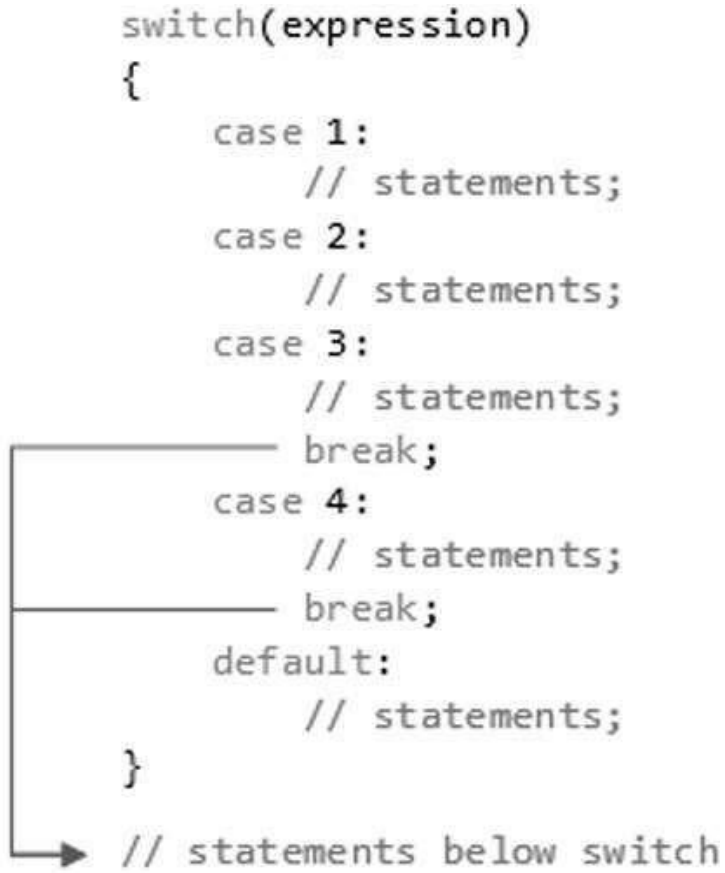
```
class ifelseifDemo
{
public static void main(String args[])
{
int i = 20;
if (i == 10)
{
System.out.println("i is 10");
}
else if (i == 15)
{
System.out.println("i is 15");
}
else if (i == 20)
{
System.out.println("i is 20");
}
else
{
System.out.println("i is not present");
}
}
}
```

```
D:\>javac ifelseifDemo.java

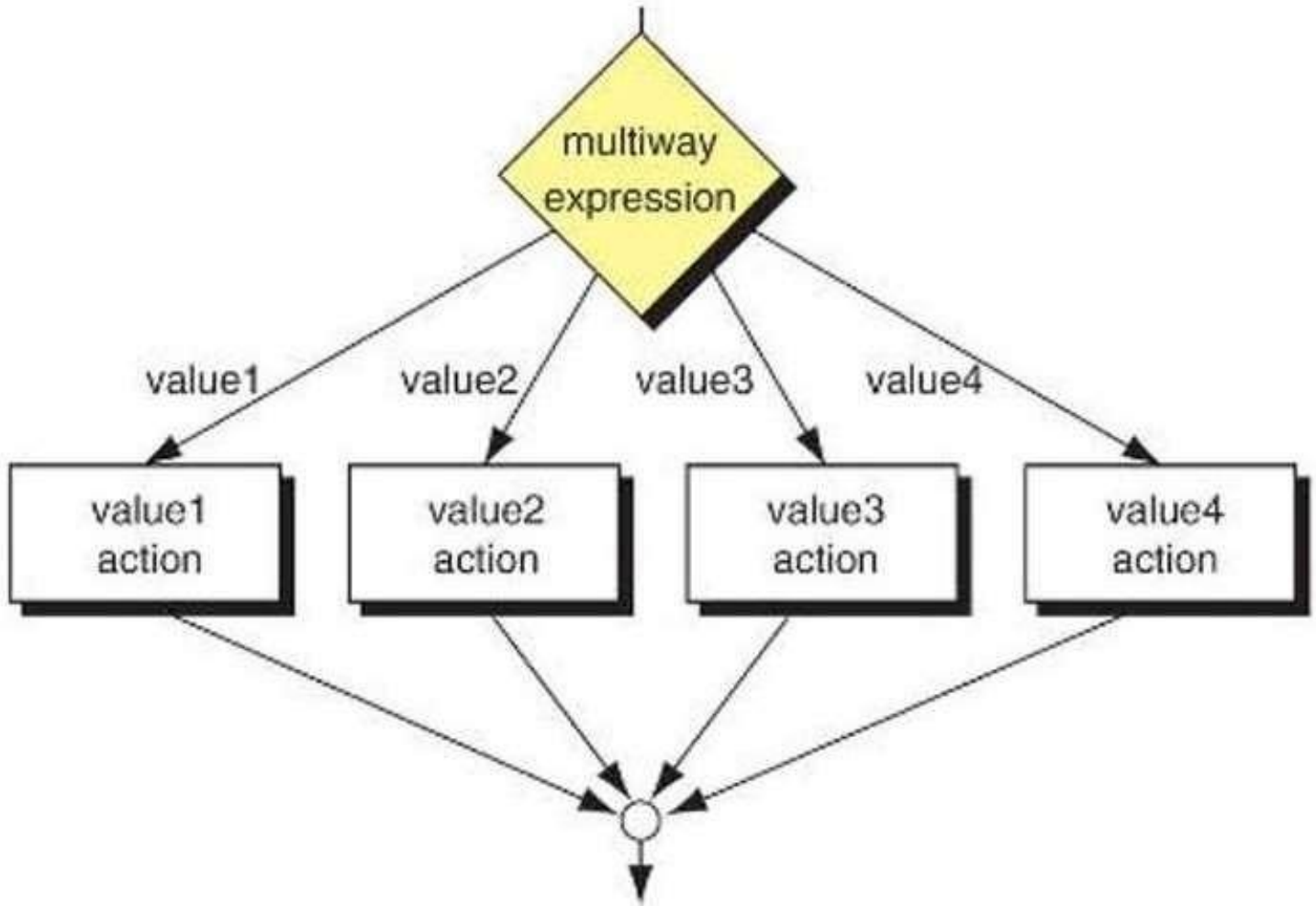
D:\>java ifelseifDemo
i is 20
```

Switch Statement:

In Java, Switch statements are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.



flowchart


multiway
expression
value1
value2
value3
value4
value1
action
value2
action
value3
action
value4
action

Example

```
class Student
{
public static void main(String[] args)
{
int num = 2;
switch (num)
 {
 case 0:
  System.out.println("number is 0");
  break;
  case 1:
  System.out.println("number is 1");
  break;
  default:
  System.out.println(“default answer”);
}
}
}
```

```
D:\JAVA LAB_AI&ML\5>javac Student.java

D:\JAVA LAB_AI&ML\5>java Student
default answer
```

2.ITERATIVE/REPETITION STATEMENTS

When ever a block of statements are used to execute repeatedly in number of times called as iterative statement.

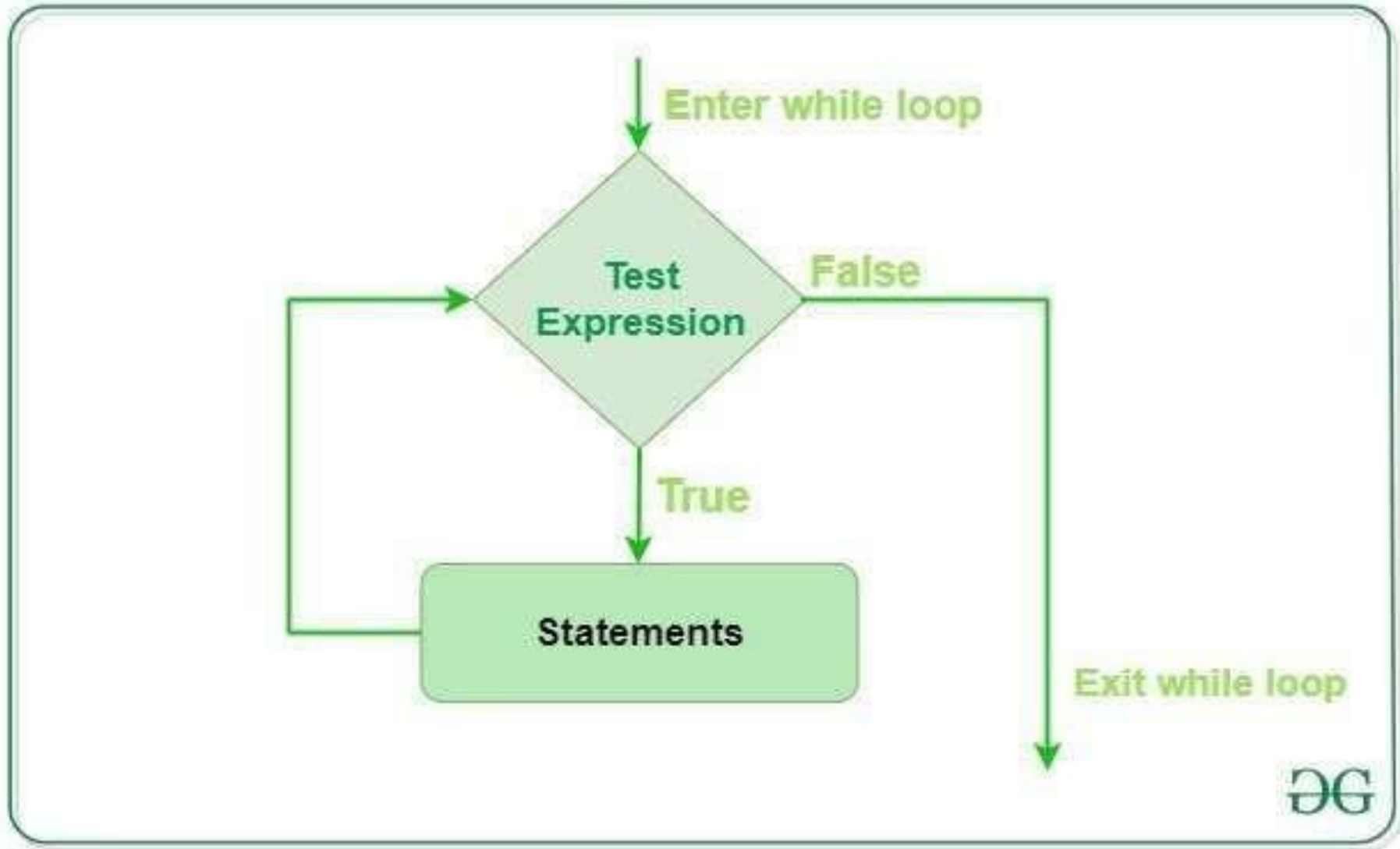
There are different types:

1.FOR LOOP

2.WHILE LOOP

3. DO-WHILE LOOP

4.JAVA FOR-EACH LOOP


Enter while loop
Test Expression
False
True
Statements
Exit while loop

a.FOR Loop:

A for loop is used to execute a block of statements repeatedly until a given condition is satisfied.

When the condition becomes false, the line immediately after the loop in the program is executed.

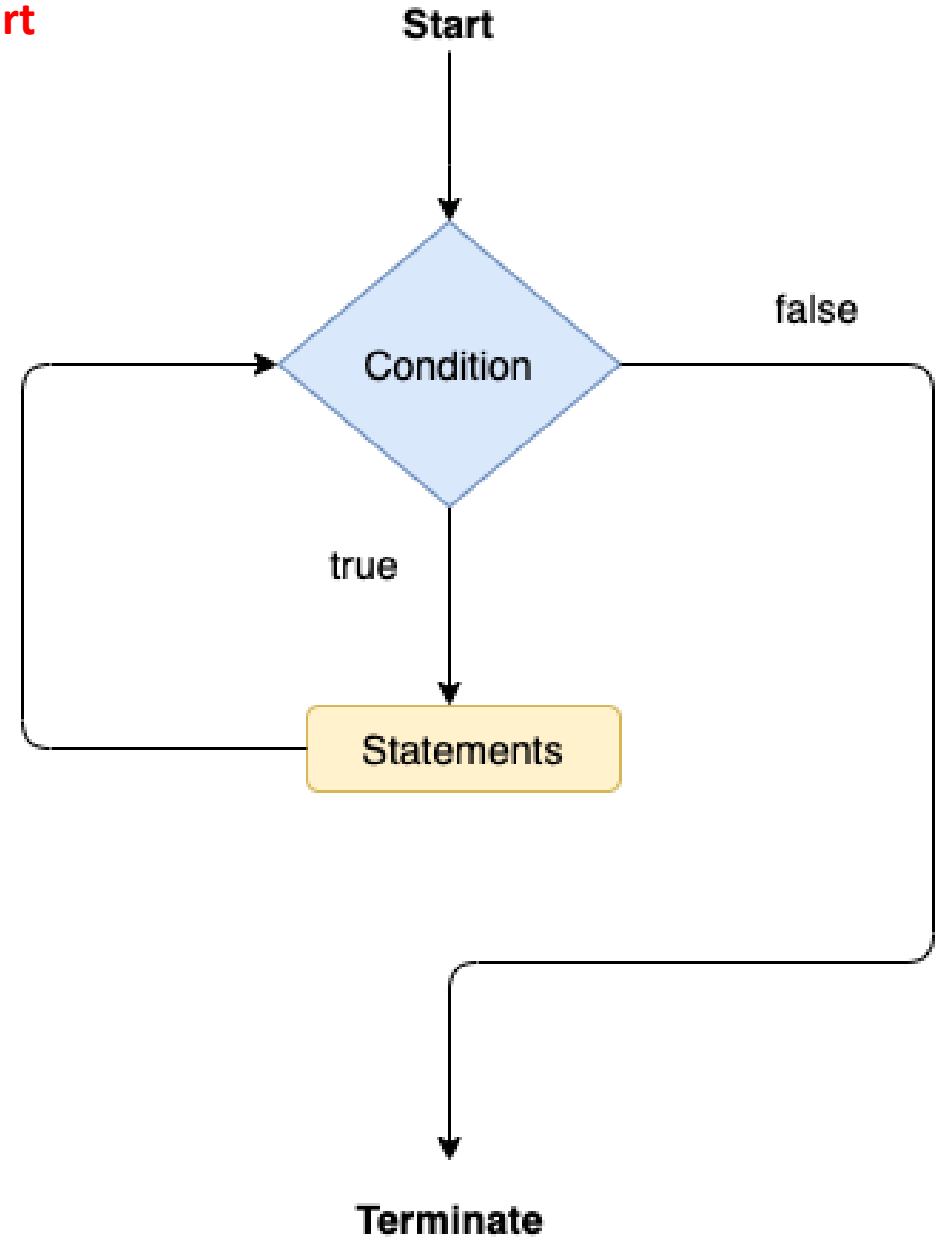
For Loop Syntax:

```
for(initialization; condition; increment/decrement)
{
block of statements ;
}
```

1.Loop Variable Initialization 2.Condition for termination

3.Updation of loop variable.

Flowchart



Example:

```
class Calculattion
{
public static void main(String args[])
{
int sum = 0;
for(int i= 1; i<=10; i++)
{
sum = sum + i;
}
System.out.println("The sum of first 10 natural numbers is " + sum);
}
}
```

```
D:\>javac Calculattion.java

D:\>java Calculattion
The sum of first 10 natural numbers is 55
```

b.While Loop:

A while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.

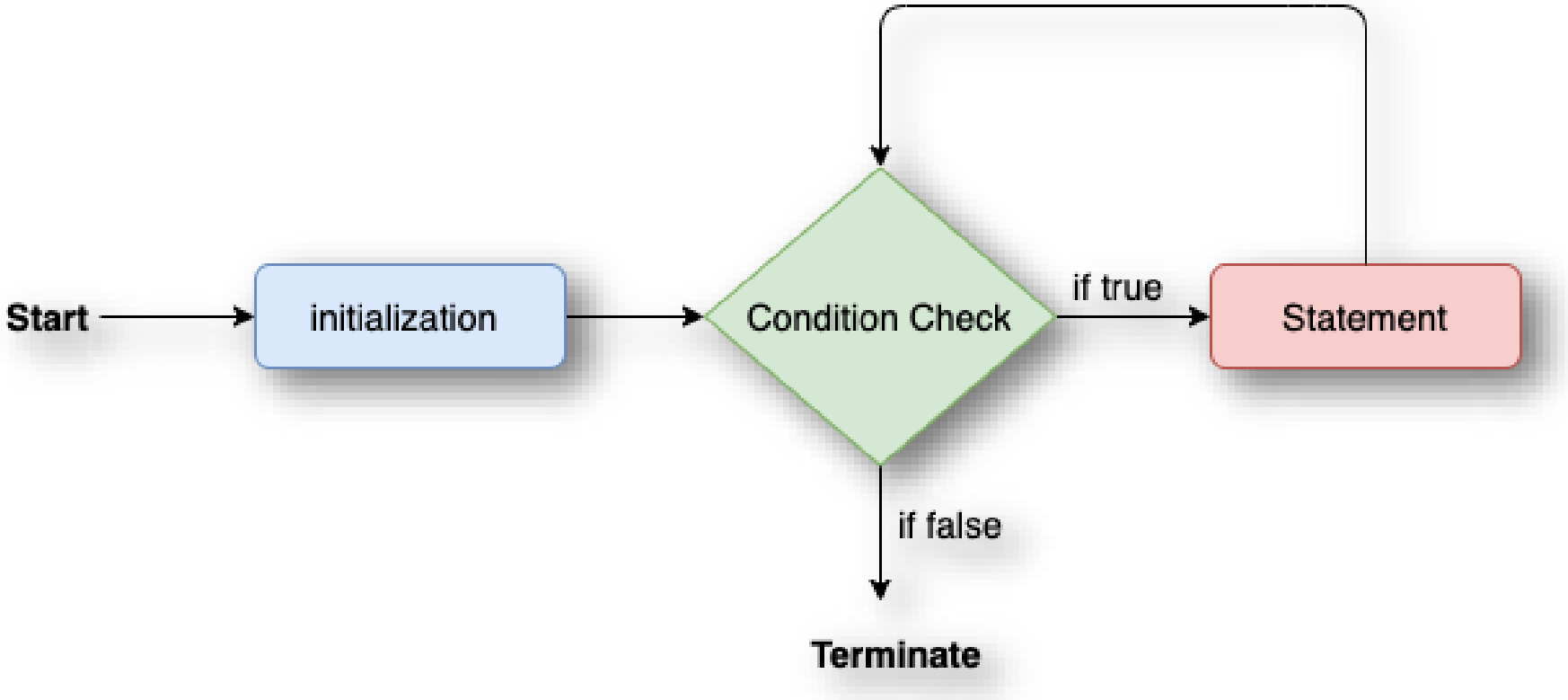
While Loop Syntax:

```
while(condition)
{
Statement block;
}
```

1.Loop Variable Initialization **2.Condition for termination**

3.Updation of loop variable.

Flowchart


Start
initialization
Condition Check
if true
Statement
if false
Terminate

```
class Calculation1
{
public static void main(String args[])
{
int i = 0;
int sum = 0;
System.out.println("Printing the list of first 10 even numbers \n");
while(i<=10)
{
sum=sum+i;
i = i + 1;
}
System.out.println("The sum of first 10 natural numbers is " + sum);
}
}
```

```
D:\>javac Calculation1.java

D:\>java Calculation1
Printing the list of first 10 even numbers

The sum of first 10 natural numbers is 55
```

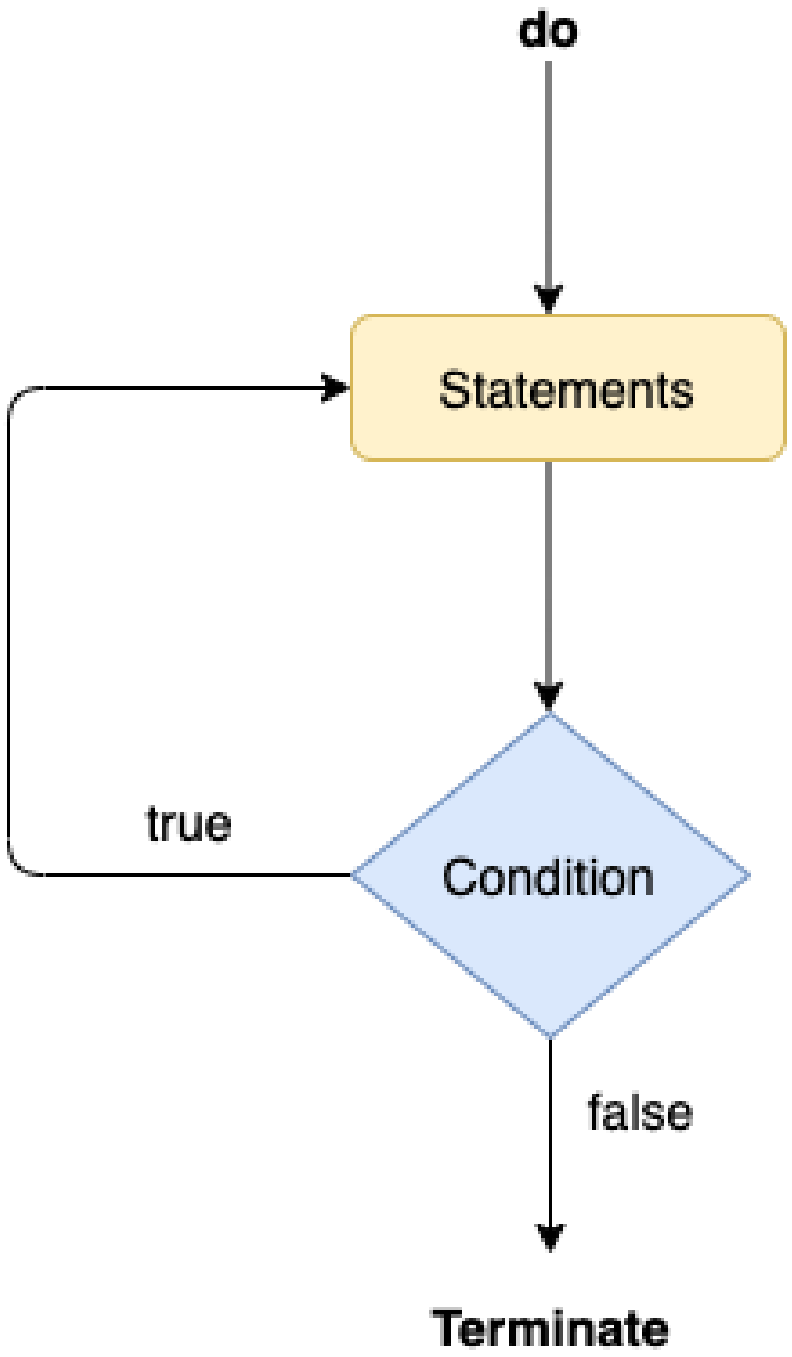
c.Java do-while loop

The do-while loop checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

Do-while Loop Syntax:

```
do
{
statements ;
}
while (condition);
```

Flowchart



Example:

```
class Calculation2
{
public static void main(String args[])
{
int i = 0;
int sum = 0;
do
{
sum=sum+i;
i = i + 1;
}
while(i<=10);
System.out.println("The sum of first 10 natural numbers is " + sum);
}
}
```

```
D:\>javac Calculation2.java

D:\>java Calculation2
The sum of first 10 natural numbers is 55
```

d.Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable.

The syntax to use the for-each loop in java is given below.

```
for (type var : array)
{
    statements using var;
}
```

- **array:** an **array** or a collection
- **var:** each item of array/collection is assigned to the **variables in Java**
- **type:** the data type of the array/collection

Example:

```
class Calculation3
{
public static void main(String args[])
{
String names[] = {"CIVIL","EEE","MECH","CSE","AIML"};
System.out.println("Printing the content of the array names:\n");
for(String branch:names)
{
System.out.println(branch);
}
}
}
```

```
D:\>javac Calculation3.java

D:\>java Calculation3
Printing the content of the array names:

CIVIL
EEE
MECH
CSE
AIML
```

3.Jump Statements

- Jump statements are used to transfer the control of the program to the specific statements. In other

- words, jump statements transfer the execution control to the other part of the program. There are

- two types of jump statements in Java, i.e., break and continue.

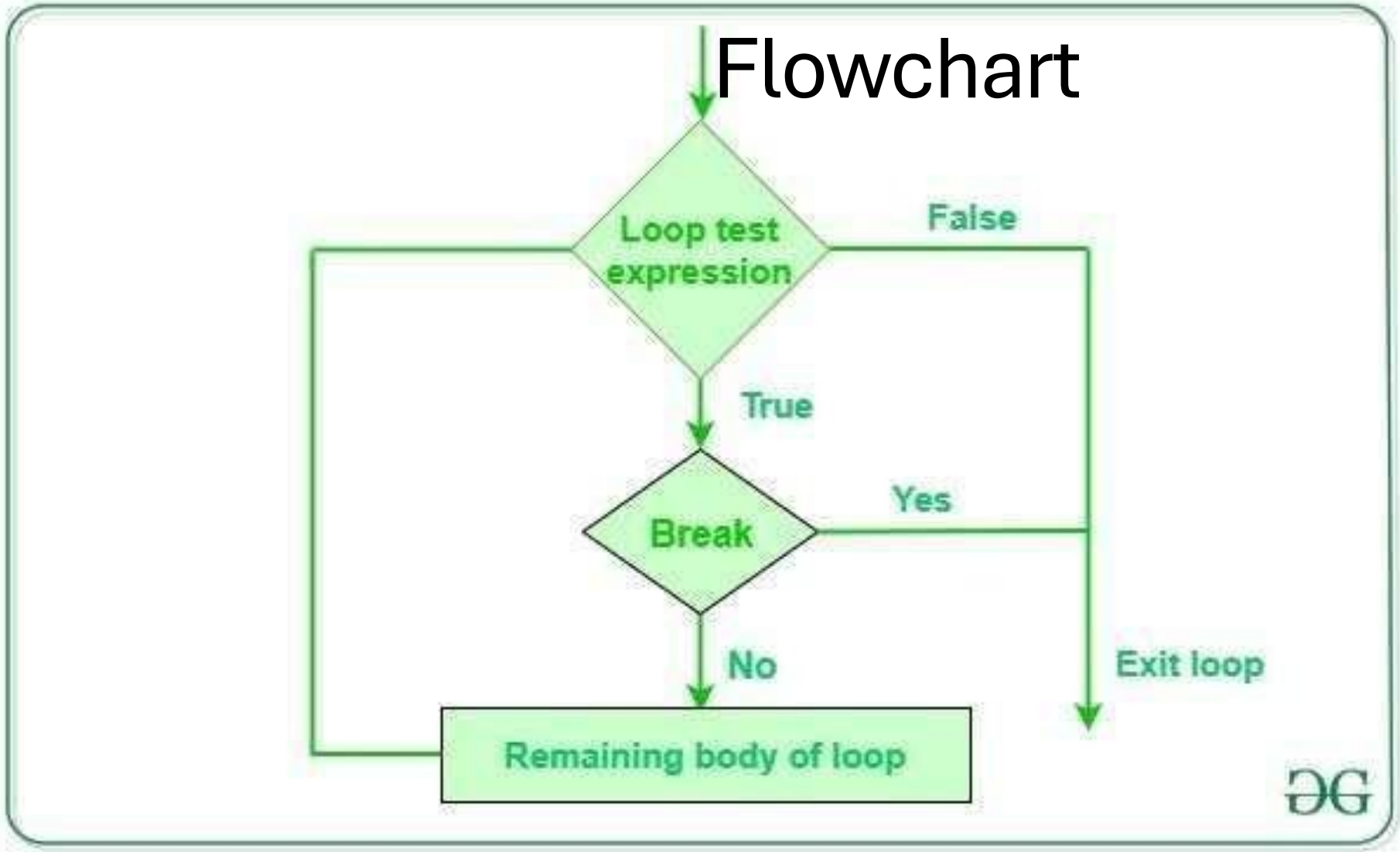
a.Java break statement

As the name suggests, the break statement is used to break the current flow of the program and transfer the control to the next statement outside a loop or statement.

However, it breaks only the inner loop in the case of the nested loop.

The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

Flowchart


Loop test expression
False
True
Break
Yes
No
Exit loop
Remaining body of loop

Example:

```
class BreakExample
{
public static void main(String args[])
{
for(int i = 0; i<= 10; i++)
{
if(i==6)
{
break;
}
System.out.println(i);
}
}
}
```

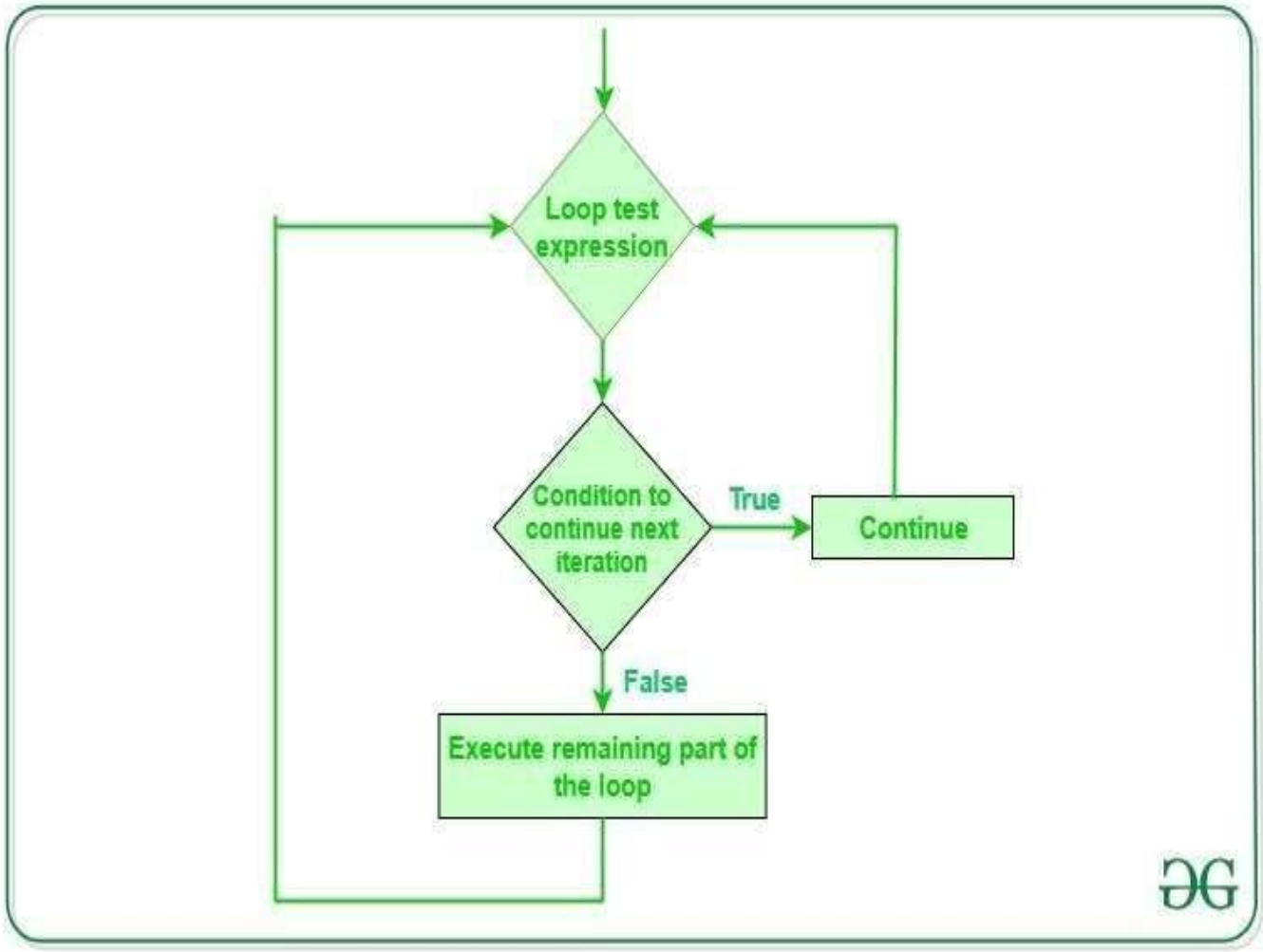
```
D:\>javac BreakExample.java

D:\>java BreakExample
0
1
2
3
4
5
```

b.Java continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

Flowchart



Example:

```
class ContinueExample
{
public static void main(String args[])
{
for(int i = 0; i<= 10; i++)
{
if(i==6)
{
continue;
}
System.out.println(i);
}
}
}
```

```
D:\>javac ContinueExample.java

D:\>java ContinueExample
0
1
2
3
4
5
7
8
9
10
```

break = EXIT 🚪
continue = SKIP ⏭️
return = EXIT METHOD

THANK YOU